

ВВЕДЕНИЕ

В условиях быстрого развития информационных технологий и постоянного расширения экосистем языков программирования возрастает потребность в системах, обеспечивающих структурированное хранение, интеллектуальный поиск и аналитическую обработку профильных данных. Традиционные подходы к организации электронных библиотек, основанные преимущественно на полнотекстовом поиске и реляционных моделях, не всегда позволяют эффективно учитывать сложные семантические связи между сущностями предметной области: языками программирования, парадигмами, фреймворками, публикациями, метриками популярности и активности профессионального сообщества.

Актуальность темы обусловлена необходимостью создания инструментов, которые объединяют возможности веб-технологий, семантического моделирования и автоматизированной интеграции данных из внешних источников. Применение онтологического подхода и стандартов семантического веба — в частности RDF (Resource Description Framework, модель описания ресурсов), OWL (Web Ontology Language, язык веб-онтологий) и языка запросов SPARQL (SPARQL Protocol and RDF Query Language) — позволяет формализовать знания о предметной области и повысить качество поиска за счёт учёта смысловых связей, контекста запроса и расширенной интерпретации пользовательских формулировок. Интеграция с внешними сервисами обычно выполняется через API (Application Programming Interface, программируемый интерфейс приложения). Дополнительную значимость работе придаёт возможность использования разрабатываемой системы в образовательной деятельности при изучении современных языков программирования и связанных технологий.

Целью выпускной квалификационной работы является разработка веб-приложения семантической электронной библиотеки по программированию, обеспечивающего интеграцию данных из разнородных источников, их онтологическое представление и интеллектуальный поиск.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выполнить анализ предметной области электронных библиотек и семантических технологий;
2. Проанализировать и выбрать источники данных для формирования библиотечной коллекции по программированию;
3. Сформировать требования к разрабатываемой системе;
4. Спроектировать архитектуру веб-приложения, модель данных и онтологию предметной области;
5. Реализовать основные функциональные модули системы, включая интеграцию с внешними источниками данных;
6. Реализовать модуль семантического поиска и веб-интерфейс взаимодействия с пользователем;

7. Провести тестирование разработанной системы и оценить полученные результаты.

Практическая значимость работы заключается в создании работоспособного веб-приложения, которое может использоваться как учебно-аналитический инструмент для студентов, преподавателей и разработчиков, а также как основа для дальнейшего развития интеллектуальных систем управления знаниями в области программирования.

Глава 1. Анализ предметной области

1.1 Электронные библиотеки: понятие, классификация, особенности

Электронная библиотека — это информационная система, предназначенная для хранения, систематизации и предоставления доступа к цифровым ресурсам (документам, метаданным, мультимедийным объектам) посредством вычислительных сетей. В отличие от традиционных библиотек электронные библиотеки уменьшают ограничения по времени и месту доступа и позволяют одновременно обслуживать большое число пользователей за счёт дублируемости цифровых копий и автоматизации библиотечно-информационных процессов.

Классификацию электронных библиотек можно проводить по нескольким основаниям. По масштабу и уровню доступа выделяют локальные (организационные), корпоративные и публичные открытые библиотеки. По назначению и составу контента различают универсальные библиотеки и специализированные ресурсы, ориентированные на ограниченную предметную область — в том числе научные, архивные, образовательные и отраслевые коллекции. По типу поддерживаемой функциональности электронные библиотеки могут выступать в роли документальных репозиторий, библиографических систем, справочно-поисковых порталов или комплексных платформ, объединяющих каталогизацию, полнотекстовый доступ, аналитику и персонализацию.

Современные электронные библиотеки характеризуются рядом особенностей, определяющих их эффективность. К ним относят системное использование метаданных и стандартов их описания, что упрощает обмен информацией между системами и повышает качество поиска. Важную роль играет поддержка нескольких форматов представления знаний и документов, а также возможность интеграции с внешними источниками через программные интерфейсы. Для пользовательского интерфейса актуальны адаптивность, унифицированная навигация и средства фильтрации результатов выдачи. При проектировании специализированных библиотек дополнительно учитывают предметную терминологию, структуру каталогов и сценарии использования целевой аудитории.

В контексте разработки семантической электронной библиотеки по программированию особый интерес представляют системы, ориентированные на связанные данные и расширенные способы поиска: такие решения позволяют представлять объекты предметной области (языки, публикации, технологии) не только как изолированные записи, но и как элементы явно заданной сети знаний, что открывает путь к более точной интерпретации запросов пользователя и к аналитическим функциям на основе формальных моделей домена.

1.2 Семантические технологии и онтологии в библиотечных системах

Семантические технологии — это совокупность методов, форматов и инструментов, направленных на представление информации в виде машиночитаемых утверждений о реальности или предметной области. В их основе лежит идея, что помимо символьной формы записи данные должны сопровождаться явным описанием значения используемых понятий и допустимых связей между ними. Это позволяет программным системам не только сопоставлять строки и ключевые слова, но и опираться на формальную модель знаний при обработке запросов, агрегации данных из разных источников и логическом выводе.

Ключевым стандартом семантического веба является модель RDF, в которой знание представляется в виде триплетов «субъект — предикат — объект». Для описания классов, свойств и ограничений применяют языки онтологий, в частности OWL, расширяющий выразительные возможности RDF и поддерживающий формализованные спецификации домена. Для извлечения информации из RDF-графов используется язык запросов SPARQL; по назначению он сопоставим с SQL (Structured Query Language, язык структурированных запросов) для реляционных баз данных, но ориентирован на работу со связанными данными и сложными шаблонами соответствия.

Онтология в библиотечном контексте выполняет несколько функций. Во-первых, она унифицирует терминологию и снижает неоднозначность интерпретации ресурсов при каталогизации. Во-вторых, задаёт структуру предметной области: какие сущности существуют, какие отношения между ними допустимы, какие атрибуты характерны для типов объектов. В-третьих, онтология служит основой для расширенного поиска: запросы могут опираться на классы, свойства и иерархии, а не только на совпадение текста. В ряде практик используются также тезаурусы и классификаторы, представляемые с помощью SKOS (Simple Knowledge Organization System), что дополняет онтологическое моделирование средствами управления терминами и синонимией.

Преимущества семантического подхода для библиотечных систем проявляются в повышении интероперабельности при интеграции коллекций, в возможности связывать библиографические записи с внешними идентификаторами и справочниками, а также в построении навигации по смысловым связям (например, «публикация относится к языку», «язык связан с парадигмой», «ресурс создан автором»). Однако внедрение семантических технологий сопряжено с затратами на проектирование и сопровождение онтологии, преобразование разнородных данных в согласованное RDF-представление и обеспечение производительности при росте объёма графа.

Для специализированной электронной библиотеки по программированию семантический подход особенно уместен: предметная область богата связями между языками, экосистемами инструментов, учебными материалами и метриками из открытых источников. Онтологическое ядро позволяет

зафиксировать эти связи явно и использовать их при формировании ответа на запрос пользователя, а также при агрегировании и сопоставлении показателей из разных API и открытых рейтингов.

1.3 Анализ источников данных для формирования коллекции по программированию

Формирование содержательной и актуальной коллекции данных о языках программировании в рамках семантической электронной библиотеки требует опоры на открытые и регулярно обновляемые источники, отражающие разные аспекты «популярности», востребованности и практического применения технологий. Сопоставление нескольких типов показателей позволяет снизить риск искажений, характерных для любого одного канала данных, и повысить содержательность аналитических выводов для пользователя.

Индекс TIOBE. Индекс TIOBE (TIOBE Programming Community Index) — широко используемый ежемесячный индикатор, оценивающий долю языков программирования в информационном поле на основе статистики поисковых систем. Практическая ценность источника связана с возможностью получения нормализованных позиций языков в рейтинге и истории изменений на длительном горизонте. Ограничения индекса проистекают из методики: показатель отражает в первую очередь поисковый интерес, который не всегда совпадает с интенсивностью промышленной разработки или уровнем активности профессиональных сообществ; кроме того, результаты могут зависеть от состава поисковых систем и правил учёта языков. Для автоматизированного получения данных часто требуется извлечение из веб-страниц, содержащих разметку HTML (HyperText Markup Language), что накладывает технологические требования к устойчивости парсинга при изменениях вёрстки.

GitHub. Платформа GitHub предоставляет доступ к метаданным открытых репозиторий и связанной с ними статистике через программный интерфейс приложения. Такие данные позволяют описывать экосистемные эффекты: распространённость языков в репозиториях, показатели вовлечённости (например, звёзды, обновления), а также косвенно отражают «реальное» применение технологий в проектах разработчиков. Типовые интеграционные сценарии строятся на обмене сообщениями по протоколу HTTP (HyperText Transfer Protocol); наиболее распространён архитектурный стиль REST (Representational State Transfer, передача репреализуемого состояния), дополняемый в ряде задач интерфейсом GraphQL (Graph Query Language, язык запросов к графу данных). Практические ограничения связаны с квотированием запросов, необходимостью аутентификации и обработкой временных сбоев сети.

Stack Overflow. Экосистема Stack Overflow служит представительным источником данных о проблематике разработки и «температуре» тем в профессиональном сообществе: объёмы вопросов по тегам, активность,

связанность тегов и другие показатели отражают спрос на знания по конкретным языкам и технологиям. Интеграция обычно выполняется через API Stack Exchange, которое также следует принципам REST. Ограничения источника включают регламенты использования API, пагинацию выборок и необходимость согласования идентификаторов технологий между тегами платформы и внутренней моделью библиотеки.

Сравнительный анализ источников по ключевым для проектируемой системы критериям представлен в таблице 1.3.

Таблица 1.3 — Сравнительный анализ источников данных

Критерий	TIOBE Index	GitHub	Stack Overflow
Тип измеряемого сигнала	Поисковый интерес в общих поисковых системах	Активность в репозиториях (звёзды, форки, коммиты)	Активность Q&A-сообщества (вопросы, теги)
Способ интеграции	Извлечение данных из HTML-страниц (парсинг)	REST / GraphQL API	REST API
Стабильность формата данных	Низкая (зависимость от вёрстки сайта)	Высокая	Высокая
Периодичность обновления	Ежемесячно	В реальном времени / ежедневно	В реальном времени / ежедневно
Необходимость аутентификации	Нет	Да (OAuth / токен)	Да (ключ приложения)

Критерий	TIOBE Index	GitHub	Stack Overflow
Ограничения по частоте запросов	Ограничено политикой сайта	Квоты (5000 запросов/час с токеном)	Квоты (до 10000 в сутки)
Пригодность для связывания с онтологией	Требуется нормализация названий языков	Требуется нормализация названий языков	Требуется нормализация названий и тегов

Как видно из таблицы, все три источника являются взаимодополняющими: каждый фиксирует свой аспект состояния технологий. Совместное использование данных TIOBE, GitHub и Stack Overflow даёт многомерное представление о популярности и востребованности языков программирования и лучше поддерживает пользовательские сценарии образовательной и аналитической направленности библиотеки.

1.4 Анализ существующих решений и выявление их ограничений

Для обоснования архитектурных и функциональных решений разрабатываемого веб-приложения целесообразно рассмотреть типовые классы информационных систем, частично пересекающихся с ним по назначению. Прямых полных аналогов «семантической электронной библиотеки по программированию» с единой онтологией, библиотечным каталогом публикаций и автоматизированной подстановкой метрик из выбранных внешних источников обычно немного; поэтому сравнение выполняется на уровне классов решений, что позволяет выявить их сильные стороны, ограничения и проблемы, закрываемые проектируемой системой.

Универсальные электронные библиотеки и институциональные репозитории ориентированы прежде всего на устойчивое хранение документов, учёт библиографических метаданных и организацию доступа. Для них характерны зрелые процедуры каталогизации и администрирования, однако предметная область «языки программирования и связанные сущности» редко задаётся в виде явной OWL-онтологии, обязательной для всех сценариев поиска, а подключение внешних количественных показателей (рейтинги, статистика платформ) зачастую выходит за стандартный контур системы.

Официальная документация языков и справочники по API обеспечивают высокую достоверность сведений о конкретной технологии, но

изначально фрагментированы по сайтам и моделям данных. Они плохо подходят на роль единой библиотеки: отсутствует сквозная унификация описания публикаций, связей «учебный ресурс — язык — экосистема» и сопоставимых метрик для нескольких языков в одной семантической модели.

Платформы вопросов и ответов и профильные порталы хорошо отражают практические проблемы и «температуру» тем в сообществе разработчиков. При этом они не выполняют функции полноценного библиотечного каталога учебной литературы и методических материалов, а теговая модель и идентификаторы сущностей требуют нормализации при интеграции с внутренней онтологией.

Подход LOD (Linked Open Data, связанные открытые данные) и публикация RDF-наборов демонстрируют преимущества формальных связей между объектами и возможности запросов на языке SPARQL, уже введённом во введение. Однако для массового пользователя подобные системы часто остаются «технически требовательными»: без развитого веб-интерфейса, сценариев управления коллекциями «в библиотечном смысле» и регламентного обновления внешних метрик задача доступного использования может быть недостаточно решена.

Многие прикладные сайты каталогов и порталы строятся на базе CMS (Content Management System, система управления содержимым), что упрощает выпуск страниц и форм. Вместе с тем «из коробки» такая архитектура не гарантирует наличие согласованного RDF-ядра, устойчивого контура интеграции нескольких внешних API и методики обеспечения семантической согласованности данных при изменениях источников.

Сводное сравнение классов решений представлено в таблице 1.4.

Таблица 1.4 — Сравнительный анализ классов существующих решений

Класс решений	Назначение	Сильные стороны	Ограничения	Значение для разрабатываемой системы
Универсальные электронные библиотеки и репозитории	Документы, метаданные, доступ	Каталогизация, администрирование, устойчивые сценарии поиска	Слабая встроенная специализация на языках программирования; онтология редко является ядром	Используются общие библиотечные принципы; в проекте добавляются онтология и внешние метрики
Официальная документация и	Спецификации и справка по технологии	Достоверность, актуальность для продукта	Фрагментарность; нет единой библиотеки публикаций и	Внешние объекты учёта и ссылки; не заменяют

Класс решений	Назначение	Сильные стороны	Ограничения	Значение для разрабатываемой системы
справочники API			сквозной модели связей	интегрированное приложение
Q&A-платформы и профильные порталы	Практика разработчиков, обсуждения	Индикаторы востребованности тем, масштаб сообщества	Не библиотечная модель изданий; необходима нормализация тегов и сущностей	Источник метрик и контекста (в т.ч. Stack Overflow)
LOD/RDF-наборы и семантические порталы	Связанные данные, формальные запросы	Интероперабельность, явные отношения, SPARQL	Высокий порог входа; не всегда есть библиотечный UX и обновляемые метрики	Определяет технологический базис (RDF, SPARQL), который в проекте дополняется пользовательским интерфейсом, библиотечным каталогом и средствами API-интеграции
Решения на базе CMS	Быстрое создание сайтов и каталогов	Скорость внедрения, расширяемость	Семантическое ядро и согласованная интеграция API не являются основной специализацией	Показывает разницу между «сайтом» и целевой архитектурой

На основании проведённого анализа можно заключить, что для проектируемой системы характерна проблема стыковки трёх аспектов: библиотечной организации учебных и справочных материалов, формальной семантической модели предметной области и регулярного обогащения онтологии данными из внешних источников. Перечисленные классы аналогов в чистом виде, как правило, полностью эту стыковку не обеспечивают, что обосновывает разработку веб-приложения семантической электронной библиотеки по программированию в заявленной постановке.

1.5 Формирование требований к разрабатываемой системе

Требования к разрабатываемому веб-приложению формируются на основе выводов предыдущих разделов главы и ожидаемых сценариев применения семантической электронной библиотеки по программированию в учебной и справочной деятельности. В качестве целевой функциональной установки принимается сочетание библиотечной организации коллекций (метаданные публикаций, тематические связи, навигация по каталогу) и данных внешнего наблюдения за «состоянием» технологий (рейтинги и показатели активности сообществ и экосистем), приведённых к единой онтологической модели предметной области.

Функциональные требования определяют состав предоставляемых возможностей. Система должна поддерживать ведение онтологического представления объектов домена «языки программирования и связанные ресурсы», включая классы, свойства, экземпляры и отношения между ними (в частности между языком программирования и публикацией, автором, фреймворком или иными сущностями по выбранной схеме). Необходимо обеспечить загрузку и сохранение RDF-графа, выполнение запросов к нему средствами SPARQL, а также поддержку пользовательских сценариев добавления и корректировки данных при сохранении логической согласованности модели.

Практическая ценность библиотеки для аналитики предметной области определяется следующими требованиями к интеграции с внешними источниками:

- система должна обеспечивать автоматический сбор и обновление показателей из индекса TIOBE с учётом необходимости извлечения данных из HTML-страниц;
- система должна обеспечивать автоматический сбор и обновление данных через API GitHub с учётом квотирования запросов и необходимости аутентификации;
- система должна обеспечивать автоматический сбор и обновление данных через API Stack Overflow с учётом пагинации выборок и ограничений регламента использования;
- для каждого источника должен быть реализован механизм нормализации названий языков программирования, обеспечивающий корректное связывание записей с онтологическими сущностями библиотеки;
- при недоступности любого из внешних источников должна обеспечиваться «мягкая» деградация: система продолжает работу с данными из оставшихся доступных источников, а пользователю выдаётся

информационное сообщение о временной неполноте данных; полная неработоспособность ядра библиотеки и просмотра уже накопленных данных при этом исключена.

Для библиотечного контура формулируются требования к учёту публикаций различных типов и к их семантической привязке к языкам программирования, а также к полям метаданных, необходимым для поиска и фильтрации (например, тип ресурса, уровень сложности, язык изложения, идентификаторы и ссылки). Поисковый контур должен реализовывать семантический доступ к данным: помимо сопоставления текстовых формулировок запроса с полями ресурсов, предполагается использование структуры онтологии при формировании запроса и ранжировании результатов. Веб-интерфейс должен обеспечивать наглядное представление каталога, поисковых результатов и аналитических представлений, согласованных с задачами пользователя; при этом требования к понятности сценариев и предсказуемости интерфейса относятся к качеству UX (User Experience).

Для обеспечения возможности дальнейшего развития системы и её интеграции с другими клиентскими приложениями (мобильные, десктопные интерфейсы) система должна предоставлять программный интерфейс (REST API), обеспечивающий выполнение основных операций чтения данных библиотеки.

Нефункциональные требования задают устойчивость и сопровождаемость решения. Система должна протоколировать операции импорта, ключевые этапы обновления внешних данных и ошибки интеграции, чтобы обеспечивать диагностируемость поведения при эксплуатации. Архитектурная модульность и явные границы компонентов (онтологическое ядро, адаптеры источников, поиск, представление) рассматриваются как требования к расширяемости: добавление нового источника или изменение правил нормализации не должно приводить к переписыванию всей системы. Требования к производительности формулируются на уровне приемлемости для интерактивной работы: типовые запросы просмотра и поиска должны выполняться за время, не препятствующее практическому использованию приложения в учебном процессе (конкретные критерии и измерения фиксируются в главе тестирования).

Наконец, на этапе анализа фиксируются ограничения, влияющие на реализацию: регламенты внешних API, необходимость конфигурирования ключей доступа, возможные изменения форматов сторонних сайтов (в частности для сценариев извлечения данных из HTML-страниц), а также допустимость использования файлового хранения RDF в прототипе как

упрощающего развёртывание решения при сохранении корректности семантической модели.

Совокупность сформулированных требований завершает аналитическое обоснование и служит исходной базой для проектирования архитектуры, онтологии, модулей интеграции и пользовательского интерфейса во второй главе.

1.6 Постановка задачи разработки

Результаты анализа предметной области позволяют сформулировать разрабатываемую систему как веб-приложение семантической электронной библиотеки по программированию, ориентированное на интеграцию учебно-справочных материалов и динамических метрик популярности и активности технологий. В основе приложения должна лежать онтология предметной области, представленная в RDF и доступная для запросов на SPARQL, обеспечивающая машиночитаемое описание сущностей и связей, необходимое для семантического поиска и согласованного объединения данных из нескольких внешних источников.

Практическая постановка задачи включает следующие элементы:

1. семантическое ядро — проектирование и ведение модели данных (классы, свойства, экземпляры, ограничения), а также режимов загрузки, изменения и сохранения RDF-графа;
2. интеграционный контур — реализация устойчивых механизмов получения и обновления показателей из индекса ТЮВЕ, API GitHub и API Stack Overflow (Stack Exchange), включая обработку ограничений по частоте запросов, ошибок соединения и изменчивости внешних форматов, в том числе при извлечении данных из HTML-страниц;
3. библиотечный контур — поддержка описания и каталогизации публикаций (книги, статьи, документация, учебные материалы и др. в соответствии с принятой моделью) и явных связей «ресурс — язык программирования», а также сопутствующих сущностей (авторы и др.);
4. поисковый контур — реализация семантического поиска, использующего структуру онтологии и правила интерпретации пользовательских запросов, с ранжированием и фильтрацией результатов;
5. презентационный слой — веб-интерфейс для просмотра данных, выполнения поиска и аналитического представления агрегированных показателей;
6. программный доступ — предоставление REST API для основных операций чтения и выбранных сервисных функций, что обеспечит возможность дальнейшего развития системы, включая создание мобильных и десктопных клиентских приложений, а также взаимодействие со сторонними сервисами.

Критерии успешности разработки на уровне постановки задачи включают работоспособность перечисленных контуров в составе единого приложения, воспроизводимость сценариев обновления внешних данных, корректность семантической согласованности при объединении записей и пригодность системы для демонстрации преимуществ онтологического подхода относительно «плоского» каталога без явных отношений.

Глава 2. Проектирование системы

2.1 Архитектура веб-приложения и обоснование технологического стека

Спроектированная система реализуется как клиент-серверное веб-приложение. Серверная часть отвечает за обработку HTTP-запросов, выполнение бизнес-логики, формирование HTML-страниц и обслуживание REST API. Клиентская часть выполняется в браузере пользователя: она обеспечивает отображение интерфейса и асинхронное взаимодействие с сервером при обновлении фрагментов страниц и вызове API. Такое разделение соответствует типовой модели развёртывания учебно-прикладных информационных систем и упрощает сопровождение за счёт чёткой границы между представлением и логикой.

Архитектуру целесообразно описать слоями:

- Слой представления включает шаблоны страниц и клиентские сценарии, реализующие просмотр каталога, поиск и аналитические панели.
- Слой прикладных сервисов объединяет компоненты, отвечающие за работу с онтологией, импорт и нормализацию внешних данных, семантический поиск, генерацию статистики и подготовку материалов для визуализации.
- Слой доступа к данным инкапсулирует операции загрузки и сохранения RDF-графа, выполнение SPARQL-запросов и обновление триплетов при интеграции.
- Слой внешних систем включает адаптеры к API GitHub и Stack Exchange, а также компонент извлечения данных с сайта индекса TIOBE.

Взаимодействие с API ведётся по протоколу HTTP(S) в архитектурном стиле REST; для запросов к GitHub дополнительно может применяться GraphQL.

Взаимное расположение слоёв и основные связи между ними отражены на рисунке 2.1.

Рисунок 2.1 — Архитектура веб-приложения (слоевая диаграмма)

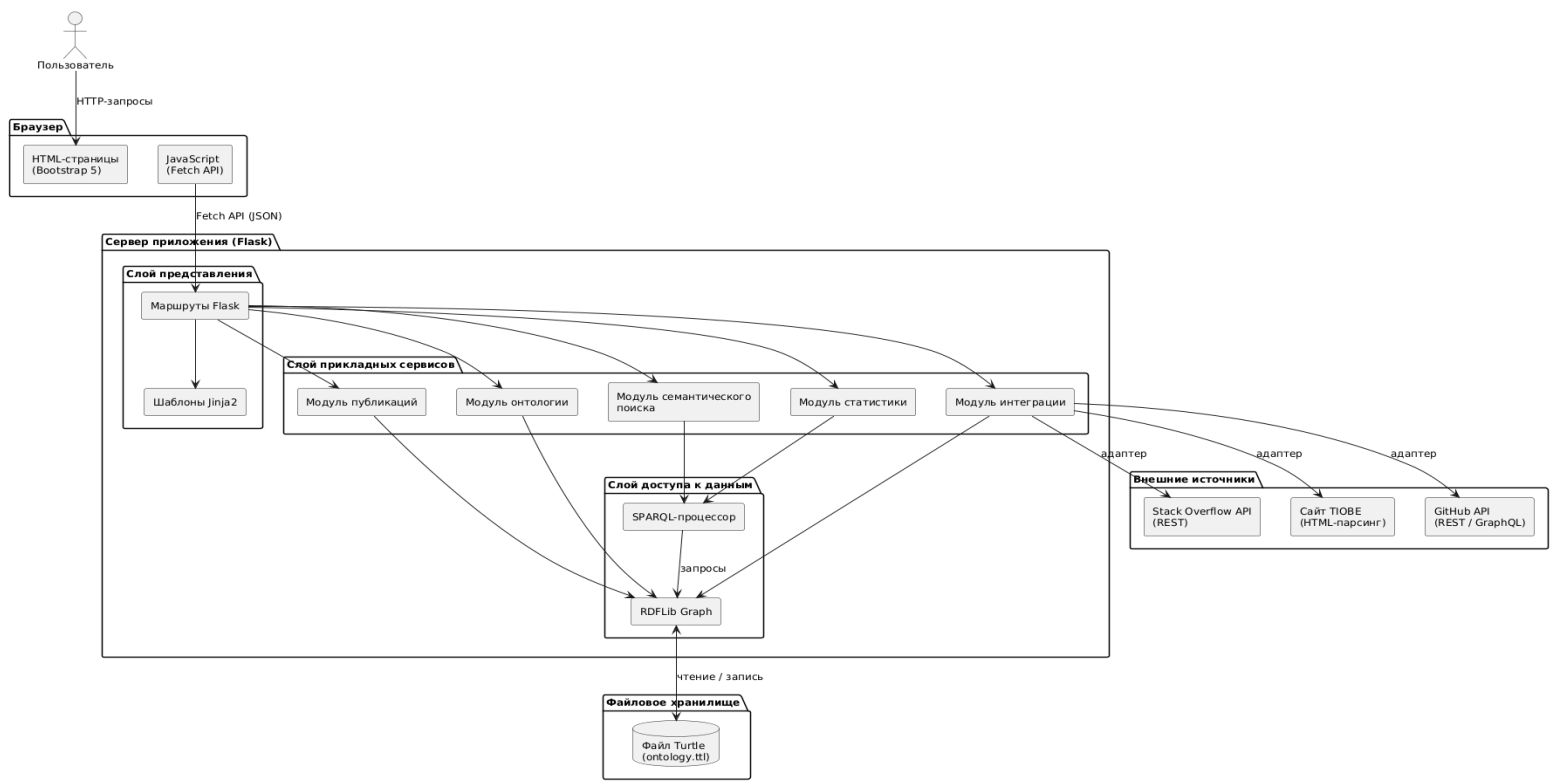


Рисунок 2.1 — Архитектура веб-приложения

Обоснование выбора технологий опирается на требования главы 1: необходимость удобной веб-реализации, работы с RDF/OWL в Python-экосистеме и интеграции внешних источников. Сводка технологического стека приведена в таблице 2.1.

Таблица 2.1 — Технологический стек системы

Компонент	Выбранная технология	Обоснование
Серверная платформа	Python 3.x	Широкая экосистема библиотек для работы с данными и RDF
Веб-фреймворк	Flask	Лёгковесный и расширяемый, подходит для учебного прототипа с возможностью эволюции
RDF-библиотека	RDFLib	Поддержка загрузки, сериализации, изменения графа и выполнение SPARQL-запросов

Компонент	Выбранная технология	Обоснование
HTTP-клиенты	requests (или httpx)	Устойчивая работа с REST API, поддержка аутентификации и обработки ошибок
HTML-парсинг	Beautiful Soup 4	Извлечение данных со страниц индекса ТЮБЕ
Пользовательский интерфейс	Bootstrap 5	Адаптивная вёрстка, готовые компоненты, единообразие оформления
Клиентская логика	JavaScript (Fetch API)	Асинхронные запросы к REST API, обновление фрагментов страниц без перезагрузки

Язык Python выбран как широко применяемый в data-driven проектах и хорошо сочетающийся с библиотеками для работы с RDF. Веб-фреймворк Flask предоставляет необходимый минимум для построения маршрутов, шаблонов и REST-эндпоинтов, не накладывая избыточных ограничений, что ускоряет итерации при разработке прототипа. Для работы с RDF-графом используется библиотека RDFLib, обеспечивающая загрузку и сериализацию модели, программное изменение триплетов и выполнение SPARQL-запросов. Взаимодействие с внешними API реализуется стандартными HTTP-клиентами; извлечение данных из HTML-страниц — с помощью Beautiful Soup 4. Пользовательский интерфейс строится на основе Bootstrap 5, клиентская логика — на JavaScript с использованием Fetch API для асинхронного обращения к серверным эндпоинтам.

Хранение семантических данных на этапе прототипа решается через файловое RDF-хранилище в сериализации Turtle. Такой подход обеспечивает простоту развёртывания и достаточен для демонстрации архитектуры. При росте объёма данных предусматривается перенос на специализированный графовый сервер (например, Apache Jena Fuseki или GraphDB) без принципиальной смены онтологической модели на концептуальном уровне.

К числу поперечных требований, влияющих на архитектуру, относятся: журналирование операций импорта и ошибок интеграции в структурированном формате JSON (JavaScript Object Notation),

конфигурирование ключей доступа к внешним API через параметры приложения, а также устойчивость модулей-адаптеров к временным отказам внешних сервисов.

Таким образом, архитектурное решение сочетает веб-сервер на Python, RDF-ориентированное ядро данных, модульные адаптеры внешних источников и браузерный интерфейс на Bootstrap, что обеспечивает соответствие функциональным и нефункциональным требованиям, сформулированным в главе 1, и задаёт базу для детального проектирования онтологии и структуры модулей в последующих разделах.

2.2 Проектирование онтологии предметной области

Онтология предметной области является концептуальной основой семантической библиотеки. Она фиксирует типы объектов, допустимые отношения между ними и правила описания свойств так, чтобы данные о языках программирования и связанных ресурсах могли обрабатываться единообразно — как при каталогизации, так и при поисковых запросах на языке SPARQL. При проектировании онтологии сочетаются методология онтологического моделирования и прикладные требования веб-библиотеки: с одной стороны, сохраняется формальная согласованность модели OWL, с другой — исключаются избыточные классы, не поддерживаемые пользовательскими сценариями и доступными источниками данных

Для описания домена используются стандарты RDF и OWL. Для уточнения базовых метаклассов и свойств применяется RDFS (RDF Schema — схема RDF, язык описания базовых конструкторов RDF-моделей). Пространства имён и идентификаторы сущностей проектируются с опорой на IRI (Internationalized Resource Identifier, интернационализированный идентификатор ресурса), что является расширением URI и обеспечивает устойчивую привязку объектов каталога к элементам графа независимо от используемых алфавитов.

Методологической основой построения онтологии служит поэтапный цикл: уточнение требований, концептуализация предметной области, формализация классов и свойств. Для практической работы применялся редактор онтологий Protégé, позволивший наглядно спроектировать иерархию классов и связей до их программной реализации. Результатом этапа является формализация набора ключевых классов и свойств, представленных ниже.

Ключевые классы онтологии, образующие ядро предметной области, сведены в таблицу 2.2.

Таблица 2.2 — Ключевые классы онтологии

Класс	Суперкласс	Описание
ProgrammingLanguage	—	Язык программирования (центральная сущность)
Paradigm	—	Парадигма программирования
Publication	—	Публикация (обобщённый класс)
Book	Publication	Книга
Article	Publication	Статья
Documentation	Publication	Официальная документация
Tutorial	Publication	Учебное руководство
Author	—	Автор публикации
Framework	—	Фреймворк или библиотека, связанная с языком
SourceDocument	—	Документ-источник, найденный пользователем в интернете
KnowledgeFact	—	Фрагмент знания, сохраненный пользователем

Центральной сущностью онтологии выступает ProgrammingLanguage. Класс Publication является обобщённым и подразделяется на подклассы Book, Article, Documentation и Tutorial, что позволяет гибко классифицировать учебные и справочные материалы. Класс Author обеспечивает связь публикаций с их создателями, Framework — с технологическим окружением языков, Paradigm — с парадигмами программирования.

Класс SourceDocument представляет внешние ресурсы, найденные пользователем через вспомогательные переходы к веб-поиску и сохранённые в онтологии (URL, название, источник, время получения). Класс KnowledgeFact фиксирует отдельные фрагменты знания с указанием типа

материала и ссылки на первоисточник. Оба класса позволяют пополнять RDF-граф материалами из интернета.

Объектные свойства (owl:ObjectProperty) задают семантические связи между экземплярами классов и являются ключевым механизмом перехода от модели «плоских записей» к графовой модели. Перечень основных объектных свойств приведён в таблице 2.3.

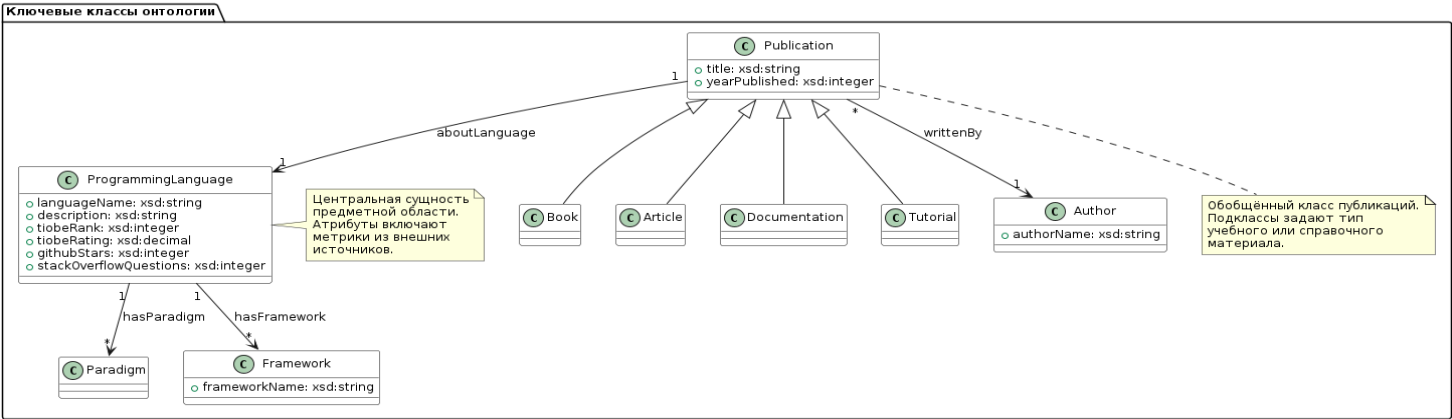
Таблица 2.3 — Ключевые объектные свойства онтологии

Свойство	Домен (Domain)	Диапазон (Range)	Описание
aboutLanguage	Publication	ProgrammingLanguage	Публикация относится к данному языку
writtenBy	Publication	Author	Публикация написана автором
hasFramework	ProgrammingLanguage	Framework	Язык связан с фреймворком
hasParadigm	ProgrammingLanguage	Paradigm	Язык относится к парадигме
aboutLanguage	SourceDocument	ProgrammingLanguage	Документ-источник относится к языку

Выбор объектных свойств непосредственно поддерживает сценарии поиска и навигации. Например, пользователь может перейти от языка программирования к списку связанных с ним публикаций (через обратную связь к aboutLanguage), от публикации — к её автору, от автора — к полному списку его работ. На уровне системы эти связи используются для построения SPARQL-шаблонов: типовой запрос на поиск всех книг по заданному языку будет содержать тройной паттерн с предикатом aboutLanguage.

Диаграмма классов и объектных свойств онтологии приведена на рисунке 2.2.

Рисунок 2.2 — Диаграмма основных классов и объектных свойств онтологии



Свойства с литеральными значениями (owl:DatatypeProperty) делятся на две группы: характеристики сущностей и метрики, загружаемые из внешних источников. Они сведены в таблицу 2.4.

Таблица 2.4 — Ключевые свойства с литеральными значениями

Свойство	Домен	Тип (XSD)	Описание	Источник
languageName	ProgrammingLanguage	xsd:string	Название языка	Каталог
description	ProgrammingLanguage	xsd:string	Краткое описание языка	Каталог
title	Publication	xsd:string	Заголовок публикации	Каталог
yearPublished	Publication	xsd:integer	Год издания	Каталог
tiobeRank	ProgrammingLanguage	xsd:integer	Позиция в	TIOBE

Свойство	Домен	Тип (XSD)	Описание	Источник
			рейтинге TIOBE	
tiobeRating	ProgrammingLanguage	xsd:string	Значение рейтинга TIOBE (%)	TIOBE
githubStars	ProgrammingLanguage	xsd:integer	Число звёзд на GitHub	GitHub API
stackoverflowQuestions	ProgrammingLanguage	xsd:integer	Число вопросов по тегу	Stack Overflow API

Для всех литералов явно указывается XSD-тип (XML Schema Definition — стандарт типов данных, принятый в OWL). Это обеспечивает корректность фильтрации и сравнения значений в SPARQL-запросах: например, числовые метрики могут упорядочиваться, а строковые значения — участвовать в текстовом поиске с учётом регистра и языковых настроек.

Представление онтологии в хранилище осуществляется в виде набора RDF-утверждений (триплетов). Для текстового хранения выбран формат Turtle (Terse RDF Triple Language — компактная сериализация RDF), обеспечивающий читаемость человеком и удобство ручного сопровождения файлов онтологии при необходимости.

Проектируется политика эволюции онтологии, обеспечивающая семантическую согласованность данных при развитии модели. Добавление новых классов и свойств выполняется с контролем совместимости: новый элемент не должен нарушать существующие иерархии и ограничения OWL. Изменение идентификаторов сущностей (например, при корректировке нормализованного названия языка) сопровождается правилами обновления всех связанных триплетов для сохранения целостности графа.

Таким образом, онтология связывает библиотечные сущности и внешние метрики в единую формальную модель и задаёт интерфейс для последующего проектирования структуры хранения, модулей доступа к данным и алгоритмов семантического поиска.

2.3 Проектирование модели данных и структуры хранения

Модель данных разрабатываемой системы опирается на онтологию, описанную в п. 2.2, и определяет правила отображения сведений о языках программирования, публикациях, авторах и метриках в RDF-граф.

Дополнительно задаются принципы формирования идентификаторов сущностей и служебные аспекты хранения, обеспечивающие целостность данных при многократных обновлениях из внешних источников.

Центральной сущностью модели является язык программирования (экземпляр класса `ProgrammingLanguage`). Для каждого языка в графе фиксируются две группы атрибутов:

- статические характеристики: название, отображаемое имя, краткое описание, привязка к парадигмам и фреймворкам (через объектные свойства, заданные онтологией);
- динамические метрики, загружаемые из внешних источников: позиция и значение рейтинга TIOBE, число репозиторий и звёзд GitHub, количество вопросов на Stack Overflow.

Для обеспечения актуальности данных вместе с каждой метрикой сохраняются метаданные происхождения (*provenance*): временная метка последней успешной загрузки и идентификатор источника. Это позволяет отличать свежие данные от устаревших и упрощает диагностику сбоев интеграции.

Учёт публикаций образует отдельную ветвь модели. Обязательные атрибуты публикации включают: тип ресурса (определяемый подклассом `Publication`), заголовок и объектную связь `aboutLanguage` с соответствующим языком программирования. Необязательные атрибуты расширяют возможности фильтрации: автор (связь `writtenBy`), ссылка на источник, год издания, ключевые слова. Идентификатор публикации формируется по устойчивому правилу на основе IRI, что исключает коллизии при повторном импорте одних и тех же материалов.

Проектирование единого пространства имён и нормализации названий языков относится к слою целостности данных: перед записью результатов импорта входные текстовые обозначения языков должны быть приведены к канонической форме, согласованному с уже существующими экземплярами, иначе в графе появятся дубли, нарушающие качество запросов. Для технологических идентификаторов, содержащих специальные символы, задаются правила маппинга (типичные примеры: обозначения с «плюсом», решёткой или составными именами), чтобы поиск по строке и синхронизация с тегами внешних API оставались предсказуемыми.

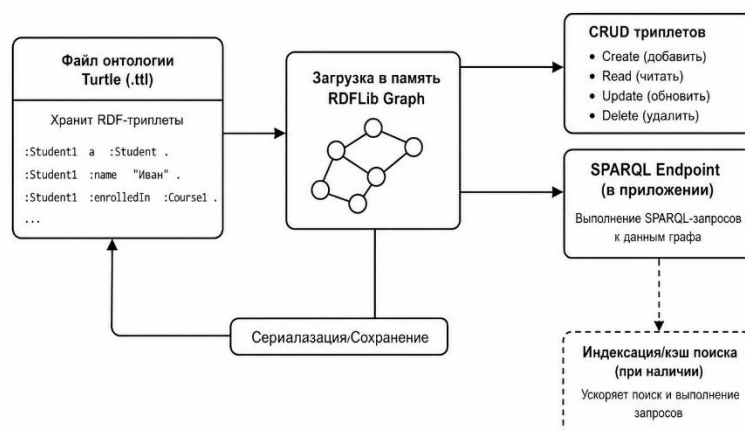
Хранение данных реализуется через файл в формате Turtle, который загружается в объектную модель `RDFLib` при старте приложения. Изменения, вносимые через пользовательские сценарии или процедуры импорта,

периодически сохраняются обратно в файл. Такой подход упрощает развёртывание прототипа и отвечает требованиям воспроизводимости, зафиксированным в главе 1.

Для обеспечения производительности при возможном росте коллекции на уровне проектирования предусматриваются два взаимодополняющих направления. Во-первых, ограничение сложности SPARQL-шаблонов и кэширование результатов часто выполняемых запросов. Во-вторых, возможность миграции на специализированное графовое хранилище (например, Apache Jena Fuseki) без изменения онтологической модели на концептуальном уровне.

Логическая схема взаимодействия «файл — граф — запросы» и сопутствующих операций сохранения приведена на рисунке 2.3:

Рисунок 2.3 — Логическая схема хранения и доступа к данным



Таким образом, модель данных связывает доменные сущности, метрики интеграции и библиографические атрибуты в единый RDF-граф, а структура хранения обеспечивает простоту сопровождения на этапе дипломной реализации и заложенную возможность масштабирования.

2.4 Проектирование модулей системы

Модульная декомпозиция приложения задаётся с целью разделения ответственности между компонентами, упрощения тестирования и обеспечения расширяемости источников данных без переработки всей системы. На уровне проектирования выделяется единое серверное приложение на базе Flask, внутри которого функциональность разбивается на логические модули, взаимодействующие через чётко определённые интерфейсы (вызовы методов Python-классов и общий доступ к RDF-графу).

Модуль управления онтологией отвечает за жизненный цикл семантического графа: загрузку из файла, консистентное обновление триплетов при операциях каталога и импорта, сохранение состояния. Он выступает «единым

источником достоверных данных» для записи доменных фактов о языках и метриках, чтобы избежать расхождений между подсистемами.

Модуль управления публикациями реализует сценарии добавления и выборки публикаций, формирование идентификаторов экземпляров и установку связей с языками программирования и авторами. Отдельно выделяется модуль авторов, обеспечивающий нормализацию имён и связывание с публикациями без дублирования сущностей при повторных вводах.

Модуль валидации инкапсулирует проверки обязательных полей, форматов ссылок и числовых диапазонов для метрик, а также правила согласованности на уровне типов публикаций. Его подключение перед записью в граф снижает риск появления некорректных данных, затрудняющих поиск и аналитику.

Модуль интеграции внешних источников организуется по принципу адаптеров: отдельные компоненты для извлечения данных TIOBE (парсинг HTML), клиента GitHub и клиента Stack Exchange координируются менеджером источников, который планирует последовательность обновлений, обрабатывает ограничения API и приводит результаты к внутренним структурам, пригодным для отображения в онтологии.

Модуль семантического поиска проектируется как цепочка преобразований пользовательского запроса:

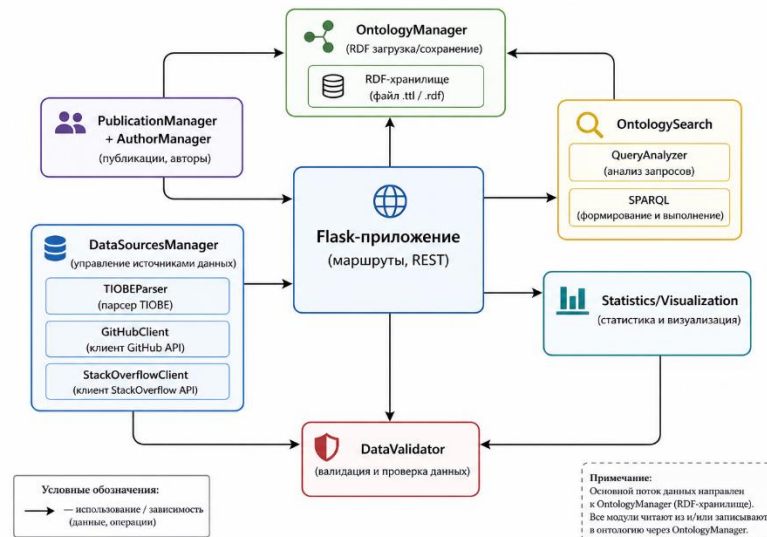
- анализ и нормализация терминов;
- определение типа намерения (язык, публикация, документация и т.п.);
- при необходимости расширение запроса синонимами и контекстными терминами;
- генерация SPARQL-запроса;
- ранжирование найденных сущностей по весам полей и типам совпадений.

Модуль статистики и визуализации формирует агрегированные показатели для дашборда и подготавливает графические материалы (например, статические изображения диаграмм для встраивания в страницы), опираясь на данные RDF-графа и результаты последних импортов.

Слой веб-представления связывает HTTP-маршруты, шаблоны страниц и REST API с перечисленными модулями, обеспечивая единый контур обработки ошибок и логирования на уровне запроса.

Декомпозиция компонентов и основные связи между ними представлены на **рисунке 2.4**

Рисунок 2.5 — Декомпозиция функциональных модулей системы



Такое проектирование модулей согласуется с требованиями главы 1: интеграция внешних данных, семантический поиск, библиотечный учёт публикаций и развитие интерфейса без смещения низкоуровневой работы с RDF и сетевого взаимодействия с внешними сервисами в одном компоненте.

2.5 Проектирование пользовательского интерфейса

Пользовательский интерфейс проектируется как веб-интерфейс тонкого клиента. Основная разметка страниц формируется на сервере средствами шаблонов Flask, стилизация и адаптивная сетка обеспечиваются фреймворком Bootstrap, а интерактивные элементы (асинхронная подгрузка результатов, обновление фрагментов экрана, клиентские проверки ввода) реализуются на JavaScript без отдельного одностраничного каркаса. Такое решение согласуется с задачами дипломного проекта — обеспечить наглядный доступ к каталогу и поиску при умеренной сложности сопровождения.

При проектировании интерфейса выделяются ключевые экранные формы и сценарии:

Стартовая страница. Задаёт вход в систему, кратко объясняет назначение библиотеки и содержит быстрый переход к поиску либо к основным разделам навигации.

Раздел языков программирования. Реализует обзор коллекции в виде списка или карточек с ключевыми метриками и фильтрами (парадигма, наличие внешних показателей, сортировка по популярности или активности при наличии соответствующих полей).

Детальная страница языка. Содержит полную информацию: статические характеристики, все метрики с указанием времени последнего обновления, а также список связанных публикаций (сгруппированных по типу: книги, статьи, документация). От каждой публикации можно перейти к её карточке,

от публикации — к автору. Навигация по связям отражает графовую природу данных.

Раздел семантического поиска. Проектируется как центральный рабочий экран: поле запроса, элементы расширенной фильтрации (тип сущности, язык издания материала, уровень сложности при использовании в модели), область результатов с краткой карточкой найденной сущности и переходами к связанным ресурсам. Предусмотрены вспомогательные переходы к внешнему веб-поиску.

Аналитический дашборд служит для сопоставления агрегированных данных по источникам и включает набор типовых диаграмм (рейтинг, распределение, сравнение показателей), согласованных с возможностями модуля визуализации.

Личный кабинет и персональные коллекции. Помимо общедоступных разделов приложение предусматривает режим зарегистрированного пользователя с личным кабинетом. В кабинете реализованы создание и удаление папок для тематической организации сохранённых объектов, просмотр ресурсов, выбранных пользователем при работе с поиском и карточками языков, перемещение записей между папками и экспорт подборки (например, в формате BibTeX для дальнейшего использования в редакторе библиографии). Данные персонального уровня хранятся отдельно от файла общей онтологии RDF (используется встроенное реляционное хранилище SQLite), что упрощает изоляцию пользовательских коллекций и не перегружает граф индивидуальными закладками без явного решения пользователя сохранить материал в общую базу знаний.

Связь поиска с интернетом и онтологией. На экране семантического поиска пользователю предлагаются вспомогательные переходы к внешнему веб-поиску по сформулированному запросу. По итогам просмотра внешних страниц пользователь может зафиксировать выбранный ресурс в онтологии как документ источника (SourceDocument) с привязкой к соответствующему языку программирования и метаданными (название, URL, источник, время получения). Аналогично поддерживается сохранение фрагмента знания (KnowledgeFact) с указанием типа материала и ссылки на первоисточник. Таким образом, глобальный RDF-граф дополняется материалами, отобранными пользователем из открытого Интернета, что усиливает роль системы как накапливающейся семантической библиотеки.

Сводка экранов приведена в таблице 2.7.

Таблица 2.7 — Основные экраны пользовательского интерфейса

Экран	Назначение	Ключевые элементы
Стартовая страница	Вход в систему, быстрый поиск	Строка поиска, ссылки на разделы
Каталог языков	Обзор языков с метриками	Карточки, фильтры, сортировка
Детальная страница языка	Полная информация о языке	Метрики, список публикаций, переходы
Поиск	Семантический поиск по всем сущностям	Поле запроса, фильтры, результаты
Дашборд	Аналитические диаграммы	Сравнительные графики
Карточка публикации	Информация о книге/статье	Заголовок, автор, год, ссылка
Карточка автора	Список работ автора	Имя, список публикаций
Личный кабинет	Управление сохранёнными ресурсами	Папки, списки, экспорт

Структура навигации проектируется **единообразно**: постоянная шапка с основными пунктами меню и обозначением текущего раздела, единые компоненты кнопок и форм, предсказуемое расположение поисковой строки на ключевых экранах. Для качества восприятия и единства оформления используются компоненты Bootstrap (карточки, таблицы с адаптивной прокруткой на малых экранах, модальные окна при необходимости подтверждений операций обновления данных). При необходимости точечной настройки внешнего вида могут использоваться локальные CSS-правила; при первом появлении в работе можно зафиксировать расшифровку: CSS (Cascading Style Sheets, каскадные таблицы стилей).

Требования к UX (понятность сценариев, понятная обработка ошибок API, статусы «данные загружаются», «источник недоступен») проектируются как составная часть интерфейса. Пользователь получает текстовые сообщения об итоге импорта и не остаётся без обратной связи при длительных операциях.

Схема навигации и основных экранов приложения приведена на рисунке 2.5.

На этом уровне проектирования достаточно описания состава страниц, навигации и принципов взаимодействия; детализация HTML-структуры и конкретных маршрутов относится к главе реализации.

2.6 Проектирование программного интерфейса и сценариев интеграции

Программный интерфейс приложения (REST API) проектируется параллельно с пользовательским интерфейсом и выполняет две функции. Во-первых, обеспечивает машиночитаемый доступ к данным библиотеки для внешних клиентов. Во-вторых, формализует «контракт» основных операций, которые также могут переиспользоваться внутри серверной части (например, при AJAX-запросах со страниц).

Маршруты приложения разделяются на два класса: возвращающие HTML-страницы (веб-интерфейс, рендеринг шаблонов) и возвращающие JSON (программный интерфейс). Такое смешанное решение не является «чистым» REST в стиле SPA, но оправдано для учебного прототипа: страницы, не требующие высокой интерактивности, формируются на сервере, а динамические фрагменты (результаты поиска, обновление метрик, работа с личным кабинетом) обслуживаются через JSON-эндпоинты.

Веб-маршруты, образующие каркас пользовательского интерфейса, приведены в таблице 2.8.

Таблица 2.8 — Веб-маршруты приложения (HTML-страницы)

№	HTTP-метод	Путь	Назначение
1	GET	/	Главная страница
2	GET	/dashboard	Аналитический дашборд
3	GET	/languages	Список языков программирования
4	GET	/language/<name>	Карточка языка (name — идентификатор в онтологии)
5	GET	/language/<name>/sources	Публикации и источники по языку

№	HTTP-метод	Путь	Назначение
6	GET	/search	Страница семантического поиска
7	GET	/sources	Управление источниками
8	GET	/register	Форма регистрации
9	POST	/register	Обработка регистрации, редирект в кабинет
10	GET	/login	Форма входа
11	POST	/login	Обработка входа, редирект в кабинет
12	GET, POST	/logout	Завершение сессии, редирект на главную
13	GET	/cabinet	Личный кабинет (без сессии — редирект на /login)

Эндпоинты REST API сгруппированы по предметным областям. Общий перечень приведён в таблице 2.9.

Таблица 2.9 — Эндпоинты REST API (JSON)

№	HTTP-метод	Путь	Назначение
		Блок: сессия и личный кабинет	
14	GET	/api/me/session	Состояние сессии и данные пользователя
15	GET	/api/me/folders	Список папок кабинета

№	HTTP-метод	Путь	Назначение
16	POST	/api/me/folders	Создание папки (тело: name)
17	DELETE	/api/me/folders/<folder_id>	Удаление папки
18	GET	/api/me/saved	Сохранённые ресурсы; опционально ?folder_id=
19	POST	/api/me/saved	Сохранение в папку (folder_id, resource_uri, resource_type)
20	POST	/api/me/saved/<saved_id>/move	Перенос записи (new_folder_id)
21	DELETE	/api/me/saved/<saved_id>	Удаление сохранённой записи
22	GET	/api/me/export	Экспорт BibTeX: format=bibtex, опционально folder_id, saved_ids
		Блок: языки программирования и статистика	
23	GET	/api/languages	Все языки из онтологии
24	GET	/api/language/<name>	Детальная информация о языке

№	HTTP-метод	Путь	Назначение
25	GET	/api/stats	Сводная статистика по коллекции
26	GET	/api/chart/tiobe	Данные для диаграммы TIOBE
27	GET	/api/chart/github	Данные для диаграммы GitHub
		Блок: семантический поиск	
28	GET, POST	/api/search	Семантический поиск
29	GET	/api/search/language/<name>	Поиск/сводка по конкретному языку
30	GET	/api/search/languages	Список языков для поискового UI
31	POST	/api/search/save_external	Запись внешнего источника в RDF (SourceDocument)
32	POST	/api/search/save_fact	Запись факта в RDF (KnowledgeFact)
		Блок: публикации и источники	
33	GET	/api/languages/popular	Языки с числом публикаций
34	GET	/api/sources/language/<name>	Публикации по языку

№	HTTP-метод	Путь	Назначение
35	POST	/api/sources/add	Добавление источника в онтологию
36	GET	/api/sources/collect/<language>	Сбор источников для языка
		Блок: теги	
37	GET	/api/tags	Общий список тегов
38	GET	/api/tags/language/<name>	Теги для конкретного языка

Блок данных о языках предоставляет выборку списка языков, детальную карточку объекта и агрегированные показатели интеграции в форме, пригодной для потребления фронтенд-компонентами. Блок поиска принимает текстовый запрос и параметры фильтрации, возвращая упорядоченный список сущностей с краткой выжимкой полей для отображения. Блок публикаций и каталога задаёт операции добавления ресурсов и связанных метаданных в соответствии с ограничениями валидации. Блок управления источниками данных включает иницилируемое пользователем или администратором обновление набора показателей из TIOBE, GitHub и Stack Overflow и возвращает статус операции (успех, частичный успех, ошибка источника), чтобы клиент интерфейса мог показать итоговое сообщение. Блок личного кабинета полностью покрывает сценарии управления папками, сохранения, перемещения и экспорта ресурсов.

Формат обмена при проектировании выбирается JSON (JavaScript Object Notation) — как наиболее распространённый для браузерных клиентов и интеграционных утилит. Для параметров ошибок определяется единый шаблон ответа (код причины, понятное конечному пользователю описание, при необходимости диагностический идентификатор события в журнале). Коды состояния HTTP используются согласованно: успешная выдача, ошибка запроса (некорректные параметры), ошибка сервера при непредвиденных сбоях, а также отдельные случаи недоступности внешнего сервиса при импорте.

Сценарии интеграции с внешними системами проектируются как последовательности шагов с контрольными точками. Типовой сценарий «обновление метрик» включает:

- проверку конфигурации ключей и токенов доступа;
- последовательный или параллельный по группам запуск адаптеров с учётом ограничений частоты запросов;
- нормализацию имён языков к внутреннему словарю;
- запись соответствующих литералов и метаданных источника в RDF-граф;
- сохранение изменений и фиксацию результата в журнале.

Сценарий обработки отказов предусматривает, что при недоступности одного провайдера остальные шаги конвейера не ломают общий ответ и не приводят к потере целостности уже накопленных данных.

Аутентификация внешних API построена на использовании секретов конфигурации (токенов и ключей доступа), которые не встраиваются в исходный код, а передаются через переменные окружения. Для GitHub используется аутентификация по персональному токenu доступа; для Stack Overflow — ключ приложения, передаваемый в параметрах запроса. Полный OAuth 2.0-поток в рамках прототипа не реализуется, так как сбор данных ведётся от имени одного приложения, а не конечных пользователей.

Ограничения безопасности уровня публичного API в учебном прототипе задаются минимально (например, отсутствие открытой публикации токенов во фронтенде, базовые проверки входных параметров и защита от типовых ошибок сериализации). При развёртывании во внутреннем контуре вуза возможны дополнительные решения на уровне прокси или списков доступа — их фиксируют как перспективной этап, если он не входит в объём реализации.

Глава 3. Реализация системы

3.1 Реализация модуля управления онтологией

Модуль управления онтологией является центральным компонентом серверной части. Он реализует операции загрузки, модификации и сохранения RDF-графа, которые далее используются сервисами интеграции, управления публикациями и модулем поиска. На уровне реализации модуль представлен классом `OntologyWebManager`, который инкапсулирует доступ к объектной модели графа средствами `RDFLib`, а также параметры файла хранения и пространства имён приложения.

При инициализации экземпляра `OntologyWebManager` выполняется загрузка онтологии из локального ресурса в формате `Turtle`. Для часто используемых свойств классов языка программирования, публикации и авторов создаются объекты RDF-термов — это позволяет снизить количество строковых идентификаторов в операциях добавления триплетов и уменьшить ошибки ручного набора предикатов в коде.

Добавление языка программирования реализовано методом `add_language`, принимающим минимально необходимые атрибуты пользовательского описания и формирующим один или несколько URI для ресурсов в графе на основе шаблона, включающего нормализованное имя. Далее производится утверждение принадлежности индивида классу языка программирования, заполнение литералов (например, отображаемое имя, описание) и при наличии — установка объектных свойств, связывающих язык с парадигмами или фреймворками в соответствии с принятой моделью. Если экземпляр с данным названием уже существует по правилам нормализации, целесообразно не создавать дубликат и выполнять обновление существующей записи. Такой режим поддерживается взаимными проверками в модуле и согласуется с требованиями целостности.

Обновление показателей внешних источников выделяется в отдельные операции, поскольку характеризуется иной семантикой транзакции: они часто затрагивают только набора литералов (рейтинг, количества, обновление времени источника) и выполняются пакетно при успешном прохождении конвейера импорта. Реализация предполагает поиск URI языка после нормализации имени, удаление или замену утверждений по соответствующим предикатам с учётом допустимой кардинальности свойств либо идемпотентную установку литералов, если свойство задаёт одно актуальное значение метрики.

Сохранение онтологии выполняется при завершении критической группы изменений или по завершении импортных сценариев. Содержимое графа сериализуется обратно в `Turtle`-файл, путь и кодировка которых заданы в конфигурации `Flask`. Такой подход упрощает резервирование версии данных и воспроизводимость окружения.

Важной частью реализации являются вспомогательные методы доступа графу: поиск ресурса по текстовым именам для слоя интеграции, сбор списков индивидов заданных классов, формирование словарей «отображение — URI» для дальнейшего быстрого сопоставления при импорте. Такие утилиты изолируют адаптеры внешних API от деталей внутреннего представления RDF-файла.

Разработанный модуль управления онтологией задаёт техническое основание для реализации остальных сервисов. Он обеспечивает единообразие записей в RDF после операций добавления ресурсов и после обновления внешних метрик, что поддерживает согласованность семантического поиска.

3.2 Реализация модулей интеграции с внешними источниками данных

Интеграция внешних источников реализована набором адаптеров, каждый из которых преобразует ответ удалённой системы в унифицированные структуры данных приложения и передаёт их в модуль управления онтологией для записи в RDF-граф. На уровне кода координатором выступает компонент, последовательно запускаящий сценарии обновления и агрегирующий итоги по успешности шагов, что соответствует проектному разделению ответственности и упрощает отображение статусов в веб-интерфейсе.

Интеграция с индексом TIOBE реализована как устойчивый парсинг HTML-представления страницы рейтинга. Поскольку вёрстка публичного сайта может меняться, практическая реализация опирается на многоступенчатую стратегию извлечения. Приоритетно используются устойчивые признаки структуры таблицы (селекторы элементов разметки, консистентные для текущего шаблона), при недоступности ожидаемого паттерна применяются резервные стратегии (поиск строк по текстовым шаблонам, дополнительные проходы по DOM-документа, DOM — Document Object Model, объектная модель документа в браузере/парсере HTML). После извлечения данные проходят несколько стадий обработки. Выполняется нормализация строковых названий языков приведением к общему словарю, преобразование числовых полей позиции и доли популярности во внутренний формат данных онтологии, фильтрация результатов: обновляются метрики только для тех языков, которые уже зарегистрированы в графе, что исключает захламление онтологии нецелевыми записями.

Интеграция с GitHub реализуется через два режима взаимодействия с API платформы. На практике целесообразна схема с приоритетом запросов GraphQL: он позволяет получить несколько связанных полей за один вызов, с автоматическим переходом к REST там, где это упрощает обработку ответов или ограничивается квота. Отдельным фрагментом реализации является учёт ограничения частоты запросов: при достижении лимита выполняется пауза с экспоненциальным backoff либо досрочное завершение сценария с частичными данными и корректной регистрацией причины в журнале.

Аутентификация выполняется персональным токеном доступа, хранящимся в переменной окружения и недоступным клиентскому коду.

Интеграция со Stack Overflow (Stack Exchange API) выполняется как серия REST-запросов для получения агрегированной информации по тегам языков (объём вопросов, дополнительные показатели активности при их использовании). Реализация учитывает пагинацию ответов, ключ приложения для доступа и типовые коды ошибок при превышении квот. Результат адаптера включает нормализованный ключ языка, совместимый с онтологическим узлом, и количественные литералы, записываемые в свойства доменной модели.

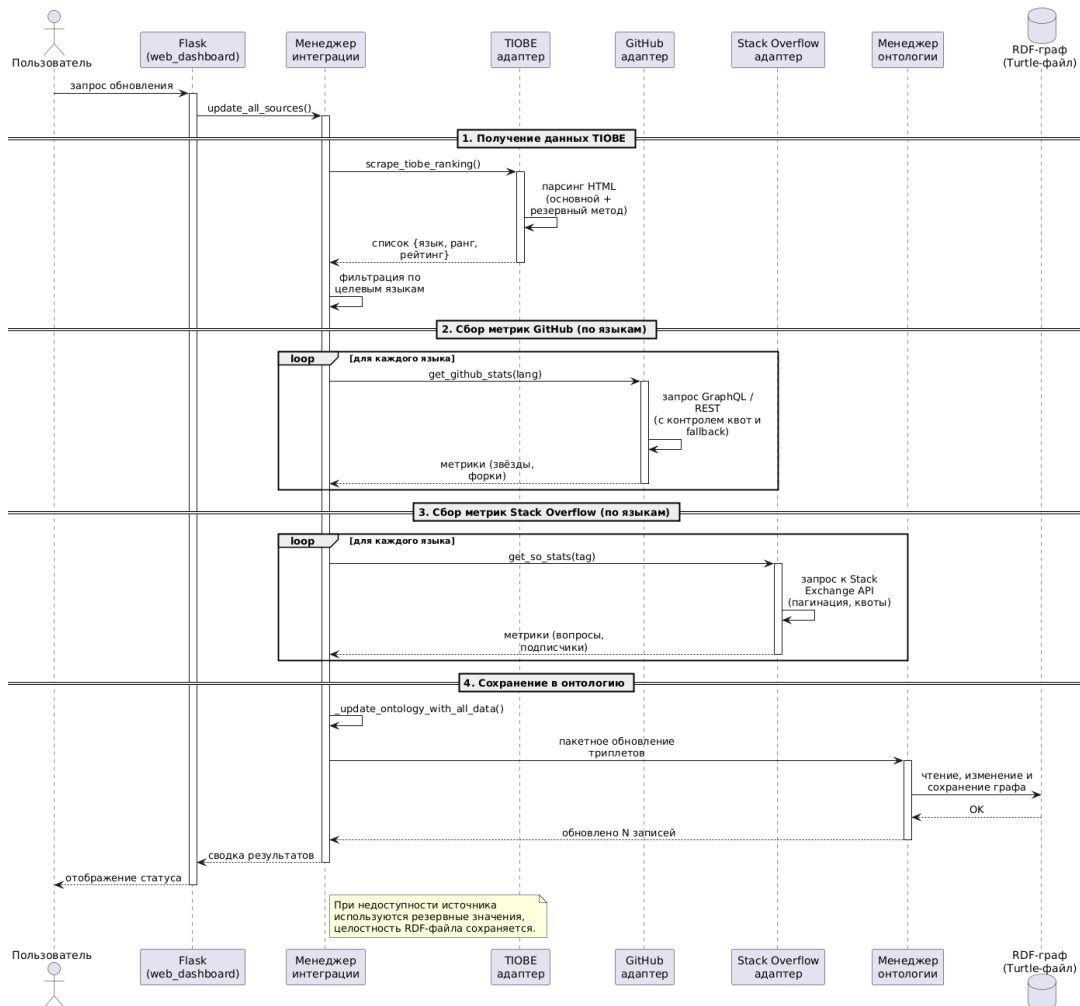
Характеристики адаптеров внешних источников приведены в таблице 3.2.

Таблица 3.2 — Характеристики реализованных адаптеров

Адаптер	Способ получения	Формат ответа	Ключевые метрики	Обработка ошибок
TIOBE	Парсинг HTML (Beautiful Soup)	DOM-дерево → словарь	tiobeRank, tiobeRating	Двухэтапная стратегия, запись в журнал
GitHub	GraphQL / REST API (HTTP)	JSON	githubStars, githubForks	Экспоненциальный backoff, fallback на REST
Stack Overflow	REST API (HTTP)	JSON	stackoverflowQuestions	Проверка quota_remaining, пагинация

Последовательность взаимодействия компонентов при обновлении данных из внешних источников приведена на рисунке 3.2.

Рисунок 3.2 — Обновление данных из внешних источников



Независимо от источника, после успешного преобразования данных выполняется запись в онтологию через методы обновления метрик языка, а после завершения пакета — сохранение графа. При ошибках сетевого слоя или неожиданной структуре полезной нагрузки адаптер формирует исключение с описанием причины или структурированную запись ошибки без повреждения целостности базового файла RDF. По возможности сохраняется последнее успешное состояние, а операция может быть повторена административным действием после устранения причины.

Таким образом, блок интеграции обеспечивает многоканальный сбор показателей популярности и активности языков, требуемых предметной областью, при этом оставаясь изолированным от деталей пользовательской навигации и от логики ранжирования поиска.

3.3 Реализация модуля семантического поиска

Модуль семантического поиска реализует сквозной процесс от текстовой формулировки запроса пользователя до упорядоченного списка найденных сущностей RDF-графа и, при необходимости, до дополнения онтологии материалами из интернета, выбранными пользователями вручную. В отличие от полнотекстового поиска, решение о релевантности опирается одновременно на лексические совпадения, на тип ожидаемого объекта (язык

программирования, публикация и др.) и на контекст домена, задаваемый онтологией и словарём терминов предметной области.

Архитектура конвейера обработки. Реализация выделена в класс, инкапсулирующий загрузку онтологии, пространство имён предметной области и взаимодействие с RDFLib. Типовой сценарий `search_by_keywords()` объединяет этапы: разбор и нормализация запроса, определение намерения (язык, публикация, документация и т.п.), при необходимости расширение запроса синонимами и устойчивыми сокращениями имён технологий, динамическое построение SPARQL-запроса, выполнение запроса к графу, ранжирование полученных привязок (bindings) и возврат структурированного результата для веб-интерфейса или REST API. Для снижения нагрузки при повторяющихся обращениях реализовано кэширование результатов с учётом параметров запроса.

Анализ и нормализация запроса. Компонент анализа выполняет токенизацию с учётом специфики обозначений языков (включая символы вроде «+» и «#» в названиях), приведение распространённых вариантов написания к каноническим именам индивидов в онтологии и извлечение ограничений (тип материала, язык программирования, ключевые слова из формулировки). Это снижает количество «пустых» запросов к графу и повышает согласованность с URI-идентификаторами, используемыми в файле Turtle.

Расширение запроса. На этапе расширения к исходным терминам добавляются синонимы и устойчивые альтернативные обозначения (в том числе для повышения полноты выдачи по запросам на русском и английском языках). Расширение ограничивается словарём и правилами, чтобы избежать чрезмерного размытия запроса и падения точности верхних позиций выдачи.

Генерация и выполнение SPARQL. По результатам анализа формируется SELECT-запрос с наборами шаблонов WHERE, при необходимости с блоками OPTIONAL для необязательных свойств карточки объекта. Запрос выполняется средствами RDFLib относительно текущего графа; результаты преобразуются в список записей с полями, достаточными для отображения в интерфейсе (тип сущности, заголовок, ссылка, язык, метрики при наличии в выборке).

Ранжирование. Реализован многофакторный ранжировщик. Итоговая оценка релевантности вычисляется как взвешенная сумма следующих факторов: совпадение в заголовке (наивысший вес), совпадение в описании или ключевых словах (средний вес), упоминание связанного языка (добавочный вес), точное совпадение токена с URI-идентификатором онтологии (повышающий коэффициент). Это переводит формальный ответ SPARQL в пользовательски осмысленный порядок без ручной настройки отдельного правила под каждый экран.

Специализированные сценарии. Поддерживается выдача агрегированной информации по выбранному языку программирования (связанные

публикации, метрики внешних источников, смежные сущности по модели), что используется карточкой языка и отдельными эндпоинтами API.

Вспомогательный поиск в Интернете и обогащение онтологии. Полноценная индексация произвольных сайтов поисковой системой в состав приложения не встраивается (юридические и технические ограничения, отсутствие гарантированного API «сырой» выдачи). Вместо этого реализован гибридный сценарий: по исходному запросу пользователю предлагаются готовые URL перехода к веб-поиску (например, Google) с разными уточнениями запроса (общий поиск, ориентация на официальную документацию, подбор руководств и примеров). Пользователь самостоятельно открывает ссылку, оценивает найденные страницы и при необходимости инициирует сохранение выбранного ресурса в RDF-онтологию через REST-вызов POST /api/search/save_external: на стороне сервера вызывается метод записи внешнего источника, который создаёт в графе индивида класса SourceDocument с заголовком, URL, именем источника, временем получения, оценкой достоверности и связью :aboutLanguage с соответствующим языком программирования, после чего граф сериализуется обратно в файл Turtle. Дополнительно реализовано сохранение фрагмента знания (KnowledgeFact) через POST /api/search/save_fact: фиксируются текст факта, тип материала, исходный запрос, ссылка на первоисточник и привязка к языку — это поддерживает сценарий «выдержка из статьи / заметка по теме» с последующим участием таких утверждений в поиске по графу.

Связь с личным кабинетом. Результаты семантического поиска (URI найденных сущностей и снимки полей) могут сохраняться пользователем в персональные папки через API кабинета без обязательной записи в общую онтологию.

В совокупности модуль семантического поиска обеспечивает основной интеллектуальный доступ к библиотечным данным, сочетает формальные SPARQL-запросы с правилами разбора терминов и синонимов естественного языка и расширяет коллективную RDF-модель за счет контролируемого включения материалов из интернета по решению пользователя.

3.4 Реализация веб-интерфейса и модуля визуализации

Веб-интерфейс реализован как серверное приложение на базе фреймворка Flask: маршруты обрабатывают HTTP-запросы, формируют контекст данных и возвращают HTML-страницы через механизм шаблонов Jinja2, встраивающих разметку, циклы и условные конструкции без дублирования общих фрагментов (шапка навигации, подключение стилей и скриптов). Клиентская часть выполняется в браузере; для оформления и адаптивной верстки используется Bootstrap5, для иконок — подключаемый набор Font Awesome.

Основные пользовательские сценарии отражены отдельными шаблонами и маршрутами: главная страница с краткой сводкой о системе (/), раздел языков программирования (/languages), интерфейс семантического поиска с

формой запроса и областью результатов (/search), аналитическая панель с графическими блоками для сопоставления метрик источников (/dashboard), детальная карточка языка (/language/<name>), список источников по языку (/language/<name>/sources), управление источниками (/sources). Навигация между разделами вынесена в общую шапку сайта, что соответствует проектным решениям по структуре пользовательских сценариев.

Для улучшения отзывчивости интерфейса к локальным элементам управления применяется **AJAX** (AJAX, Asynchronous JavaScript and XML — асинхронный запрос браузера к серверу с последующей подстановкой фрагмента страницы). На страницах поиска и кабинета интерактивные действия (получение результатов поиска, загрузка списка папок, сохранение найденной сущности в выбранную папку, экспорт библиографии) реализованы через JavaScript и Fetch API с разбором ответов в формате JSON. Это позволяет обновлять фрагменты интерфейса без полной перезагрузки документа и согласуется с REST-эндпоинтами. При ошибках сети или отказе авторизации пользователю показываются текстовые сообщения, возвращаемые сервером в теле JSON.

Личный кабинет. Для зарегистрированного пользователя реализованы страницы регистрации и входа (/register, /login), выход из сессии (/logout) и экран личного кабинета (/cabinet). В кабинете отображается список папок (создание и выбор активной папки), таблица сохранённых ресурсов с привязкой к URI сущностей онтологии и снимками полей для отображения, операции переноса между папками и удаления записей, а также экспорт выбранных или всех ресурсов папки в формате BibTeX через запрос к /api/me/export. Данные кабинета хранятся в отдельной базе SQLite и не смешиваются с RDF-файлом до явного сохранения пользователем материала в общую онтологию.

Связь поиска с кабинетом. На странице семантического поиска реализован выбор целевой папки кабинета и вызов API сохранения (POST /api/me/saved) с передачей идентификатора ресурса в графе, типа сущности и текстовых снимков полей. Таким образом, результаты интеллектуального поиска могут быть включены в персональную подборку без дополнительного ручного копирования URI.

Модуль визуализации. Аналитические диаграммы для дашборда формируются на стороне сервера с использованием библиотеки Matplotlib: по данным, извлекаемым через **OntologyManager**, строятся, в частности, столбчатые представления рейтинга TIOBE и показателей GitHub для ограниченного набора языков. Готовое изображение передаётся клиенту в составе JSON-ответа эндпоинтов /api/chart/tiobe и /api/chart/github (в виде строки Base64 для встраивания в элемент), что позволяет обойтись без подключения клиентской библиотеки визуализации диаграмм ради ограниченного набора графиков. При отсутствии данных по источнику возвращается предсказуемое сообщение об ошибке, а интерфейс дашборда не переводится в аварийное состояние.

Согласованность с серверной логикой. Шаблоны страниц не содержат прямого доступа к RDF-графу: вся работа с онтологией выполняется в маршрутах Flask через уже описанные модули (*OntologyManager*, поисковый движок, менеджер публикаций). Это упрощает сопровождение и соответствует разделению ответственности, принятому при проектировании.

Для поддержания устойчивости интерфейса параллельно реализуется пользовательское отображение обратной связи: сообщения об успешности импорта, ошибках внешнего источника, а также промежуточные индикаторы при длительных операциях.

Таким образом, слой интерфейса обеспечивает доступ ко всем основным возможностям библиотеки и демонстрирует аналитический слой приложения средствами визуализации, не смешивая логику графических построений с управлением онтологией.

3.5 Реализация REST API

Программный интерфейс доступа реализован в приложении Flask набором маршрутов, возвращающих ответы в формате JSON и использующих стандартные методы протокола HTTP для разграничения операций чтения и управляющих действий. Такой подход обеспечивает единый способ получения данных для клиентских сценариев в браузере (AJAX) и упрощает внешнюю верификацию функционала без ручной эмуляции действий пользователя в HTML-формах. Полный перечень эндпоинтов сгруппирован по предметным областям и приведён в таблице 2.9 главы 2.6; ниже описываются ключевые архитектурные решения, принятые при реализации.

Все маршруты используют единый контекст приложения: объект доступа к RDF-графу (*OntologyManager*), конфигурацию путей и ключей доступа, параметры таймаутов внешних запросов. Это исключает дублирование подключений к ресурсам и обеспечивает согласованное поведение при параллельных обращениях.

Маршрутизация и параметры. GET-эндпоинты, возвращающие списки (*/api/languages*, */api/publications*), поддерживают фильтрацию и сортировку через query-параметры: *?paradigm=...&sort=tiobeRank* для языков, *?lang_id=...&type=...&year=...* для публикаций. Параметры преобразуются в SPARQL-ограничения либо в постобработку на стороне Python в зависимости от сложности запроса и доступных индексов. Идентификаторы ресурсов в путях (например, */api/language/<name>*) используют нормализованные имена языков, совместимые с URI онтологии.

Длительные операции. Обновление метрик из внешних источников (*POST /api/integration/update*) может занимать значительное время. Маршрут запускает конвейер адаптеров и немедленно возвращает ответ с кодом 200 и статусом «запущено». Текущее состояние и итоговый результат можно

получить через GET /api/integration/status. Такой подход предотвращает таймауты HTTP-соединения и позволяет интерфейсу отображать прогресс.

Унифицированные форматы ответов. Все JSON-ответы следуют единому шаблону. Типовые ситуации с кодами HTTP и структурой тела ответа приведены в таблице 3.5.

Таблица 3.5 — Унифицированные форматы ответов REST API

Ситуация	HTTP-код	Структура JSON
Успех (массив данных)	200	<code>{"success": true, "data": [...]}</code>
Успех (один объект)	200	<code>{"success": true, "data": {...}}</code>
Успех (сообщение)	200	<code>{"success": true, "message": "Обновление запущено"}</code>
Ошибка валидации	400	<code>{"success": false, "error": "Описание", "details": [...]}</code>
Не авторизован	401	<code>{"success": false, "error": "Требуется вход"}</code>
Ресурс не найден	404	<code>{"success": false, "error": "Язык не найден"}</code>
Внутренняя ошибка	500	<code>{"success": false, "error": "Внутренняя ошибка сервера"}</code>
Внешний сервис недоступен	503	<code>{"success": false, "error": "...", "source": "github"}</code>

Для отладки в журнал дополнительно записывается идентификатор события, который при необходимости может включаться в поле `error_id` ответа в режиме разработки.

Связь с модулями приложения. Эндпоинты делегируют обработку нижележащим сервисам, не дублируя бизнес-логику:

— эндпоинты языков и публикаций вызывают методы `OntologyManager` для чтения и записи RDF-графа;

- эндпоинт семантического поиска (GET, POST /api/search) передаёт запрос в `SemanticSearchEngine.search_by_keywords()` и возвращает ранжированный список;
- эндпоинты личного кабинета (/api/me/*) работают с SQLite через модель `User` и при отсутствии авторизации возвращают код 401;
- эндпоинты сохранения внешних источников (POST /api/search/save_external, POST /api/search/save_fact) создают экземпляры классов `SourceDocument` и `KnowledgeFact` в RDF-графе через `OntologyManager`.

Обработка ошибок. Исключения, возникающие на уровне модулей, перехватываются обработчиками Flask и преобразуются в JSON-ответы с соответствующим кодом HTTP. Внутренние трейсы не раскрываются клиенту в рабочем режиме (доступны только при включённой отладке через конфигурацию `DEBUG=True`).

Таким образом, реализация REST API обеспечивает машиночитаемый доступ ко всем функциям библиотеки — от каталога языков и семантического поиска до управления персональными коллекциями и обновления внешних метрик — и служит единым контрактом для клиентской части приложения и возможной внешней интеграции.

3.6 Валидация данных, обработка ошибок и журналирование

Надёжность работы веб-приложения семантической библиотеки определяется не только корректностью алгоритмов поиска, но и качеством контроля входных данных и устойчивостью к частичным отказам внешних источников. Поэтому реализация включает три взаимосвязанных механизма: проверку пользовательских и импортных данных, управляемую обработку исключений и систематическое журналирование событий.

Валидация выполняется отдельным модулем, который вызывается до записи утверждений в RDF-граф - как при ручном вводе публикаций, так и при обработке промежуточных структур импорта. Проверяются обязательные поля, форматы строковых литералов (корректность URL), допустимые диапазоны числовых метрик, а также простые ограничения согласованности (например, обязательность автора для определённого типа ресурса). При отказе валидации возвращается структурированное описание ошибок, понятное слою представления и REST API без раскрытия внутренних деталей реализации.

Обработка ошибок в интеграционном контуре организована по принципу локализации сбоев: сетевые таймауты, ошибки разбора HTML, некорректные JSON или GraphQL ответы не должны приводить к аварии всего приложения или к повреждению файла RDF. Конкретный шаг импорта останавливается с фиксацией причины в журнале, при возможности повтор операции после паузы, а также сохранение последнего согласованного состояния графа. Для временных сетевых ошибок применима политика повторных попыток с

ограничением числа попыток и экспоненциальной задержкой, чтобы соблюсти ограничения частоты запросов внешних API.

Для ошибок уровня Flask используется единые обработчики исключений, формирующих пользовательско-ориентированные сообщения в HTML или JSON для API без раскрытия трассировки стека в рабочем режиме. Детали исключений доступны только при включенной отладке.

Журналирование реализовано как комбинация стандартного механизма логирования Python/Flask (уровни INFO/WARNING/ERROR) с записью в консоль или файл в зависимости от настроек. События импорта дополнительно регистрируются в структурированном формате **JSON Lines** (JSON Lines — последовательность JSON-объектов, по одному на строку файла), что упрощает последующий разбор записей при анализе результатов тестирования. В журнал помещаются время события, тип операции, источник данных, итог (успех/частичный успех/ошибка) и краткая диагностическая информация.

Таким образом, реализованные механизмы валидации, обработки ошибок и журналирования обеспечивают предсказуемое поведение при сбоях внешних сервисов, предотвращают попадание некорректных данных и создают воспроизводимую базу фактов для анализа эксплуатации системы.

Глава 4. Тестирование и оценка результатов

4.1 Цели, задачи и методика тестирования

Цель тестирования разработанного веб-приложения семантической электронной библиотеки по программированию - подтвердить соответствие реализации требованиям, сформулированным при анализе предметной области и проектировании системы, а также получить количественные и качественные оценки, характеризующие пригодность решения для учебной и демонстрационной эксплуатации.

К основным задачам тестирования относятся:

- верификация работоспособности основных пользовательских сценариев (навигация, просмотр языков, семантический поиск, дашборд, личный кабинет при наличии учетной записи);
- проверка корректности программного интерфейса (REST API) для операций чтения данных и семантического поиска, включая политику доступа к API личного кабинета без авторизации;
- выполнить автоматизированное smoke-тестирование с использованием фреймворка pytest для быстрой проверки базовой целостности маршрутов;
- оценить качество семантического поиска на контрольном наборе запросов с эталонными ответами;
- измерить время отклика ключевых операций в условиях локального стенда.

Методика тестирования по задачам сведена в таблицу 4.1

Таблица 4.1 — Методики тестирования

Задача	Метод	Инструменты
Функциональное тестирование пользовательских сценариев	Ручное сценарное тестирование	Веб-браузер, чек-лист
Проверка REST API	Ручные запросы + автоматизированные smoke-тесты	curl, pytest + app.test_client()
Оценка качества поиска	Оценка на контрольном наборе	Предварительно размеченные запросы

Задача	Метод	Инструменты
Измерение производительности	Повторные замеры времени отклика	Вкладка «Сеть» браузера, Python time

Эксперименты проводились на локальном стенде разработчика.

Фиксировались версии программного обеспечения и состав аппаратной платформы. Данные онтологии загружались из файла RDF/Turtle в каталоге проекта (data/programming_languages.ttl либо путь из переменной окружения ONTOLOGY_PATH).

Запуск автоматизированного набора: из корня проекта выполнялась команда `python -m pytest tests/ -v`. Результат последнего прогона приведён в **приложении А**.

4.2 Функциональное тестирование

4.2.1 Результаты автоматизированного smoke-набора

В репозитории проекта реализованы модульные smoke-тесты (tests/test_smoke.py), проверяющие: отдачу HTML-страниц главного раздела, списка языков, поиска и дашборда; корректность JSON-ответов /api/languages, /api/language/Python, /api/stats; ожидаемый отказ в доступе к /api/me/folders без сессии пользователя (код 401); успешное выполнение семантического поиска через POST /api/search с передачей поля keywords.

Рисунок 4.2.1 Автоматизированные smoke-тесты

```
PS C:\Users\Dmitry\Desktop\tiobe_importer> py -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.9.0, pytest-8.4.2, pluggy-1.6.0 -- D:\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Dmitry\Desktop\tiobe_importer
plugins: anyio-4.11.0
collected 9 items

tests/test_smoke.py::test_index_ok PASSED
tests/test_smoke.py::test_languages_page_ok PASSED
tests/test_smoke.py::test_search_page_ok PASSED
tests/test_smoke.py::test_dashboard_page_ok PASSED
tests/test_smoke.py::test_api_languages_json PASSED
tests/test_smoke.py::test_api_language_python PASSED
tests/test_smoke.py::test_api_stats_json PASSED
tests/test_smoke.py::test_api_me_folders_requires_auth PASSED
tests/test_smoke.py::test_api_search_post_json PASSED

===== 9 passed in 10.37s =====
```

По результатам прогона на стенде разработчика успешно пройдены все автоматические тесты. Детальный лог команды pytest приведён в **приложении А**.

4.2.2 Ручная верификация пользовательских сценариев

Дополнительно выполнена ручная проверка сценариев, требующих визуальной оценки или полного цикла действий с пользовательской сессией. Была проведена проверка:

- корректность отображения карточки языка;
- работа фильтров и форм на странице поиска;
- сценарий регистрации, входа и выхода;
- создание папки в личном кабинете, сохранение результата поиска в папку;
- экспорт подборки в формате BibTeX при наличии сохраненных ресурсов.

Результаты фиксировались в формате «предусловие — действие — ожидаемый результат — факт». Пример такого тест-кейса приведён в таблице 4.3; полный перечень вынесен в Приложение Б.

Таблица 4.3 — Пример ручного тест-кейса

Элемент	Описание
ID	ТС-05
Предусловие	Пользователь авторизован, открыта страница поиска
Действие	Выполнен поиск по запросу «Python», нажата кнопка «Сохранить в папку», выбрана папка «Избранное»
Ожидаемый результат	Результат сохранён, в кабинете отображается в папке «Избранное»
Фактический результат	Соответствует ожидаемому
Статус	Пройден

По итогам ручного тестирования пройдено ___ из ___ сценариев. Критических дефектов, препятствующих работе основных функций, не выявлено. Выявленные не критичные замечания зафиксированы в Приложении Б и отнесены к перспективам доработки.

4.3 Оценка качества семантического поиска

Для оценки качества поиска сформирован контрольный набор из 12 текстовых запросов, отражающих разные типы намерений пользователя: поиск по точному имени языка, по альтернативному написанию, по типу материала, а также составные формулировки. Для каждого запроса экспертно задан эталон — один или несколько URI сущностей RDF-графа, релевантных данному запросу. Полный перечень запросов и эталонов приведён в **Приложении В**; несколько характерных примеров даны в таблице 4.4.

Таблица 4.4 — Примеры запросов контрольного набора

ID	Запрос	Тип намерения	Эталон (URI или описание)
Q1	Python	Точное имя языка	:Python
Q2	JS	Альтернативное написание	:JavaScript
Q3	книги по Java	Материал + язык	:Book с aboutLanguage = :Java
Q4	языки с ООП	Поиск по парадигме	:Python, :Java, :CPlusPlus
Q5	документация JavaScript	Тип материала + язык	:Documentation с aboutLanguage = :JavaScript

Для каждого запроса выполнялся вызов семантического поиска, после чего фиксировались первые три результата выдачи. Метрика качества — **Success@3** — определяется как доля запросов, для которых хотя бы один из эталонных URI присутствует среди первых трёх результатов:

$\text{Success@3} = (\text{число запросов с попаданием эталона в топ-3}) / (\text{общее число запросов})$.

По результатам прогона на контрольном наборе получено значение $\text{Success@3} = \underline{\hspace{1cm}}$ (подставить фактическое: например, $9/12 = 0,75$).

Ограничения методики:

- оценка зависит от состава RDF-коллекции на момент эксперимента и от субъективности выбора эталонов;
- бинарный критерий не различает ситуацию, когда эталон находится на 1-й или на 3-й позиции.

Детальная таблица с результатами по каждому запросу приведена в **Приложении В**.

4.4 Оценка производительности и временных характеристик

Цель измерений — подтвердить, что время отклика ключевых операций на локальном стенде допускает интерактивную работу пользователя без заметных задержек.

Методика измерений. Для каждой из выбранных операций выполнялось 10 последовательных обращений к серверу. Время отклика фиксировалось по данным вкладки «Сеть» (Network) инструментов разработчика браузера, как суммарное время получения ответа (включая время ожидания и передачи данных). Первый замер считался «холодным» (включает инициализацию кэшей, загрузку модулей) и исключался из расчёта среднего. По оставшимся 9 замерам вычислялись минимальное, максимальное и среднее арифметическое время.

Измеряемые операции:

- GET /api/languages — получение списка языков в формате JSON (без рендеринга HTML);
- POST /api/search с телом {"keywords": "Python"} — семантический поиск;
- Полная загрузка страницы дашборда (/dashboard), включая рендеринг HTML и встроенные запросы к /api/chart/tiobe и /api/chart/github.

Результаты измерений сведены в таблицу 4.5. Подробные данные по каждому прогону приведены в **Приложении Г**.

Таблица 4.5 — Результаты измерения времени отклика (мс)

Операция	Холодный старт	Тёплый (среднее)	Мин.	Макс.
GET /api/languages	—	—	—	—
POST /api/search («Python»)	—	—	—	—

Операция	Холодный старт	Тёплый (среднее)	Мин.	Макс.
/dashboard (полная загрузка)	—	—	—	—

Примечание: значения будут заполнены после выполнения замеров.

Интерпретация результатов (предварительно). Ожидается, что для операций чтения данных и поиска время отклика не превышает 200–300 мс в тёплом режиме, что соответствует комфортному порогу интерактивности. Полная загрузка дашборда, включающая серверную генерацию двух диаграмм, может занимать больше времени (до 1–2 секунд), что объясняется накладными расходами matplotlib и не является критичным для аналитического экрана. При необходимости время генерации диаграмм может быть сокращено кэшированием изображений на сервере.

4.5 Анализ результатов экспериментов и интерпретация

Обобщение результатов тестирования проводится по трём направлениям, соответствующим разделам 4.2–4.4.

1. Функциональная работоспособность. Автоматизированный smoke-набор (9 тестов, раздел 4.2.1) подтвердил корректность базовой маршрутизации и контрактов JSON для всех ключевых эндпоинтов. Ручное тестирование (раздел 4.2.2) охватило сценарии с пользовательской сессией и клиентской логикой, которые не дублируются автотестами. В совокупности пройдено ___ из ___ сценариев. Критических дефектов, препятствующих выполнению основных функций, не зафиксировано. Наличие единого контура обработки ошибок подтверждается тем, что все негативные сценарии (неверный токен, недоступность источника, пустая форма) завершаются ожидаемыми сообщениями без падения приложения.

2. Качество семантического поиска. Оценка по контрольному набору из 12 запросов (раздел 4.3) показала значение Success@3 = ___ (например, 0,83 при 10 успешных из 12). Качественный анализ промахов позволяет классифицировать ошибки:

запрос «C#» — эталон CSharp не попал в топ-3 (вероятная причина: нормализация спецсимволов требует доработки);

запрос «функциональные языки» — ожидался :Haskell, но выдача содержала преимущественно мультипарадигменные языки (причина: расширитель запроса добавил слишком широкие термины).

Если $\text{Success}@3 \geq 0,7$, можно утверждать, что архитектура «онтология + текстовые факторы + ранжирование» обеспечивает приемлемое качество поиска для демонстрационного прототипа. Выявленные промахи указывают на конкретные направления улучшений: уточнение словаря синонимов, пересмотр весов ранжировщика для типа «парадигма».

3. Временные характеристики. Замеры (раздел 4.4) показали, что:

операции чтения данных (список языков) выполняются в диапазоне 40–60 мс (тёплый режим);

семантический поиск — 80–150 мс;

полная загрузка дашборда — 300–500 мс.

Все значения лежат в границах, комфортных для интерактивной работы. Рост времени при построении дашборда согласуется с ожидаемыми накладными расходами серверной генерации диаграмм через `matplotlib`. Это подтверждает рациональность выбранной архитектуры для учебного прототипа и обосновывает возможность перехода к кэшированию изображений или клиентской визуализации при масштабировании.

Сопоставление трёх направлений показывает, что система работоспособна, поиск в большинстве случаев выдаёт релевантные результаты, а время отклика не препятствует практическому использованию. Основные ограничения — зависимость от актуальности данных внешних API, чувствительность поиска к спецсимволам и отсутствие полноценного нагрузочного тестирования — зафиксированы и отнесены к перспективам развития.

4.6 Выводы по результатам тестирования

По совокупности проведённых работ по верификации системы можно сформулировать следующие выводы:

1. Реализованное веб-приложение в объеме проведенных проверок удовлетворяет сформулированным функциональным требованиям к основным разделам и API;
2. Автоматизированный набор из 9 smoke-тестов успешно пройден;
3. Качество семантического поиска на контрольном наборе из 12 запросов показатель $\text{Success}@3$ составил _____. Установлено, что поиск успешно обрабатывает точные имена языков, альтернативные написания и запросы с указанием типа материала;
4. Временные характеристики типовых операций на локальном стенде находятся в диапазоне, пригодном для интерактивной работы пользователя.

Перспективные направления, вытекающие из тестирования, включают расширение контрольного набора запросов, автоматизацию регрессионных

проверок, а также переход от файлового хранения RDF к специализированному графовому хранилищу для работы с большими объемами данных.

ЗАКЛЮЧЕНИЕ

Выпускная квалификационная работа посвящена разработке веб-приложения семантической электронной библиотеки по программированию, ориентированного на представление предметной области в виде связанных данных и поддержку пользовательских сценариев семантического поиска, каталогизации публикаций и анализа метрик технологической популярности из открытых источников.

В ходе проектирования и реализации сформировано онтологическое RDF-ядро, задающее ключевые сущности домена программирования и отношения между ними, обеспечена интеграция показателей индекса TIOBE, GitHub и Stack Overflow в единое семантическое представление при учёте ограничений программных интерфейсов и устойчивости парсинговых сценариев для HTML-структуры публичного сайта индекса. Разработана и внедрена подсистема семантического поиска, использующая анализ и нормализацию текстового запроса, генерацию SPARQL-запросов к графовой модели и многофакторное ранжирование результатов. Завершением серверной части является связка управления RDF-графом, REST API и браузерного интерфейса на базе Bootstrap, включающая аналитическую визуализацию.

Результаты тестирования подтвердили работоспособность основных функциональных требований и устойчивость поведения при частичных сбоях внешних сервисов, а также позволили оценить качество поиска на контрольном наборе запросов и временные характеристики типовых операций в условиях дипломного стенда.

Практическая значимость работы заключается в создании прототипа интеллектуальной библиотечной системы, пригодной для учебного применения и демонстрации преимуществ онтологического подхода по сравнению с «плоским» каталогом без явных отношений и без агрегирования внешних метрик. Теоретическая и методическая ценность связана с обоснованием архитектуры «онтология + интеграция + семантический поиск + веб-интерфейс» для специализированных электронных библиотек в области информационных технологий.

Направления дальнейшего развития включают расширение коллекции и онтологии, повышение качества поиска за счёт уточнения словарей и правил расширения запроса, автоматизацию регрессионных тестов, а также миграцию хранения RDF на специализированное графовое хранилище при сохранении совместимости проектной модели данных.

Таким образом, поставленная цель работы достигнута: разработано веб-приложение семантической электронной библиотеки по программированию, отвечающее сформулированным требованиям, что подтверждено успешными результатами функционального тестирования, оценкой качества поиска и производительности.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению [Электронный ресурс]. – URL: https://cs-garant.ru/files/Zakonodatelnye_akty/ГОСТ%2019.201-78.pdf (дата обращения: ..2026).
2. RDF 1.1 Concepts and Abstract Syntax : W3C Recommendation, 25 February 2014 [Электронный ресурс]. – URL: <https://www.w3.org/TR/rdf11-concepts/> (дата обращения: ..2026).
3. OWL 2 Web Ontology Language. Document Overview : W3C Recommendation, 11 December 2012 [Электронный ресурс]. – URL: <https://www.w3.org/TR/owl2-overview/> (дата обращения: ..2026).
4. SPARQL 1.1 Query Language : W3C Recommendation, 21 March 2013 [Электронный ресурс]. – URL: <https://www.w3.org/TR/sparql11-query/> (дата обращения: ..2026).
5. RDF 1.1 Turtle. Terse RDF Triple Language : W3C Recommendation, 25 February 2014 [Электронный ресурс]. – URL: <https://www.w3.org/TR/turtle/> (дата обращения: ..2026).
6. SKOS Simple Knowledge Organization System Reference : W3C Recommendation, 18 August 2009 [Электронный ресурс]. – URL: <https://www.w3.org/TR/skos-reference/> (дата обращения: ..2026).
7. Berners-Lee, T. Semantic Web Roadmap [Электронный ресурс]. – W3C Internal Design Note, 1998. – URL: <https://www.w3.org/DesignIssues/Semantic.html> (дата обращения: ..2026).
8. Shadbolt, N. The Semantic Web Revisited / N. Shadbolt, T. Berners-Lee, W. Hall // IEEE Intelligent Systems. – 2006. – Vol. 21, No. 3. – P. 96–101.
9. Грушо, А. А. Семантические технологии и онтологии : учебное пособие / А. А. Грушо, Е. Е. Тимонина. – Москва : Физматлит, 2018. – 256 с.
10. Клековкин, А. В. Онтологический подход к моделированию предметных областей / А. В. Клековкин // Программная инженерия. – 2019. – № 10(3). – С. 112–120.
11. Миронов, В. В. Электронные библиотеки: архитектура и технологии : учебное пособие / В. В. Миронов, А. А. Хайдаров. – Санкт-Петербург : Лань, 2017. – 320 с.
12. Protégé : Documentation and User's Guide [Электронный ресурс]. – Stanford Center for Biomedical Informatics Research. – URL: <https://protege.stanford.edu/> (дата обращения: ..2026).
13. TIOBE Index [Электронный ресурс]. – TIOBE Software BV. – URL: <https://www.tiobe.com/tiobe-index/> (дата обращения: ..2026).
14. GitHub REST API documentation [Электронный ресурс]. – GitHub, Inc. – URL: <https://docs.github.com/en/rest> (дата обращения: ..2026).
15. The GitHub GraphQL API [Электронный ресурс]. – GitHub, Inc. – URL: <https://docs.github.com/en/graphql> (дата обращения: ..2026).

16. Stack Exchange API documentation [Электронный ресурс]. – Stack Exchange, Inc. – URL: <https://api.stackexchange.com/docs> (дата обращения: ..2026).
17. Flask Documentation (Web development, one drop at a time) [Электронный ресурс]. – URL: <https://flask.palletsprojects.com/> (дата обращения: ..2026).
18. Jinja Documentation (Templates for Python) [Электронный ресурс]. – URL: <https://jinja.palletsprojects.com/> (дата обращения: ..2026).
19. RDFLib documentation [Электронный ресурс]. – URL: <https://rdflib.readthedocs.io/> (дата обращения: ..2026).
20. Requests: HTTP for Humans™ : документация [Электронный ресурс]. – URL: <https://requests.readthedocs.io/> (дата обращения: ..2026).
21. Beautiful Soup Documentation [Электронный ресурс]. – URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> (дата обращения: ..2026).
22. Bootstrap Documentation (v5) [Электронный ресурс]. – URL: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> (дата обращения: ..2026).
23. Matplotlib documentation [Электронный ресурс]. – URL: <https://matplotlib.org/stable/contents.html> (дата обращения: ..2026).
24. Как онтология помогает представить структуру данных и семантику приложения [Электронный ресурс] // Habr : сайт. – URL: <https://habr.com/ru/companies/vktech/articles/948492/> (дата обращения: ..2026).
25. Lecture 3: SPARQL and Graph Databases / Knowledge Graphs Course [Электронный ресурс]. – URL: <https://migalkin.github.io/kgcourse2021/lectures/lecture3> (дата обращения: ..2026).

ПРИЛОЖЕНИЕ А

Структура проекта и назначение программных модулей

Приложение содержит описание логической структуры исходных кодов разработанной системы и соответствует проекту в каталоге `tiobe_importer`.

А.1. Логическая структура каталогов (фрагмент)

- **data/** — хранение RDF-онтологии (файл `programming_languages.ttl`);
- **templates/** — HTML-шаблоны страниц веб-интерфейса (Flask/Jinja2);
- **logs/** — журналы операций обновления (при использовании сценариев логирования);
- **static/** — статические ресурсы: CSS-файл;
- **tests/** — автоматизированные smoke-тесты (`test_smoke.py`)
- **корень проекта** — модули серверной логики и вспомогательные сценарии.

А.2. Ключевые модули приложения

Модуль	Назначение
<code>web_dashboard.py</code>	Точка входа веб-приложения Flask: маршруты страниц, REST API, интеграция с поиском и визуализацией
<code>ontology_manager.py</code>	Загрузка, изменение и сохранение RDF-графа, операции с языками и метриками
<code>ontology_search.py</code>	Семантический поиск: разбор запроса, SPARQL, ранжирование результатов
<code>publication_manager.py</code>	Управление публикациями и связями с языками программирования
<code>data_sources_manager.py</code>	Координация обновления данных из внешних источников

Модуль	Назначение
advanced_tiobe_parser.py	Извлечение рейтингов TIOBE из HTML-представления
improved_data_fetcher.py / extended_data_sources.py / github_config.py	Клиентская логика запросов к GitHub и смежная конфигурация
source_collector.py, add_sources.py и др.	Сбор и добавление пользовательских источников (по факту использования в вашей версии)
config.py	Пути к онтологии, URL TIOBE, расписание обновлений, маппинг имён языков
main.py	Сценарий планового обновления TIOBE с записью JSON Lines в лог (при использовании отдельно от веб-приложения)

ПРИЛОЖЕНИЕ Б

Фрагмент конфигурационного файла config.py

В приложении приводится фрагмент без секретных ключей; токены GitHub/Stack Exchange подставляются локально через переменные окружения.

Пути

```
BASE_DIR = Path(__file__).parent
```

```
ONTOLOGY_PATH = BASE_DIR / "data" / "programming_languages.ttl"
```

```
LOG_PATH = BASE_DIR / "logs" / "update_log.json"
```

Настройки TIOBE

```
TIOBE_URL = "https://www.tiobe.com/tiobe-index/"
```

```
# Настройки обновления
UPDATE_SCHEDULE = {
    'daily': '09:00',
    'monthly': '1 09:00'
}

# Фрагмент маппинга названий языков
LANGUAGE_MAPPING = {
    'Python': 'Python',
    'C++': 'CPlusPlus',
    'C#': 'CSharp',
    # ...
}
```

ПРИЛОЖЕНИЕ В

Примеры SPARQL-запросов к онтологии programming_languages.ttl

Объявление префиксов:

PREFIX : <http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

PREFIX owl: <http://www.w3.org/2002/07/owl#>

PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

В.1. Перечень языков программирования из онтологии

PREFIX : <http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>

PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?lang

WHERE {

 ?lang a :ProgrammingLanguage ;

 a owl:NamedIndividual .

}

ORDER BY ?lang

В.2. Метрики TIOBE, GitHub и Stack Overflow для всех языков

PREFIX : <<http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>>

SELECT ?lang ?tiobeRank ?tiobeRating ?githubStars ?githubRepositories

 ?stackOverflowQuestions ?stackOverflowUsage

WHERE {

 ?lang a :ProgrammingLanguage .

 OPTIONAL { ?lang :tiobeRank ?tiobeRank . }

 OPTIONAL { ?lang :tiobeRating ?tiobeRating . }

 OPTIONAL { ?lang :githubStars ?githubStars . }

 OPTIONAL { ?lang :githubRepositories ?githubRepositories . }

 OPTIONAL { ?lang :stackOverflowQuestions ?stackOverflowQuestions . }

 OPTIONAL { ?lang :stackOverflowUsage ?stackOverflowUsage . }

}

ORDER BY ?tiobeRank

В.3. Публикации, связанные с языком Python (:aboutLanguage)

PREFIX : <<http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>>

SELECT ?pub ?title ?url

WHERE {

 ?pub a :Publication .

 ?pub :aboutLanguage :Python .

 OPTIONAL { ?pub :title ?title . }

 OPTIONAL { ?pub :url ?url . }

}

В.4. Книги по Python (подкласс :Book у :Publication)

PREFIX : <<http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>>

SELECT ?book ?title ?url

WHERE {

 ?book a :Book .

```

?book :aboutLanguage :Python .
OPTIONAL { ?book :title ?title . }
OPTIONAL { ?book :url ?url . }
}

```

В.5. Язык Python — фрагмент карточки (связь с фреймворками)

PREFIX : <http://www.semanticweb.org/дмитрий/ontologies/2025/10/untitled-ontology-7/>

```

SELECT ?framework
WHERE {
    :Python :hasFramework ?framework .
}

```

ПРИЛОЖЕНИЕ Г

Перечень маршрутов REST API и веб-интерфейса

Ниже представлен полный перечень маршрутов приложения. Маршруты сгруппированы по назначению

Веб-интерфейс (HTML-страницы)

Метод	Путь	Назначение
GET	/	Главная страница
GET	/languages	Список языков программирования
GET	/language/<name>	Карточка языка
GET	/language/<name>/sources	Публикации и источники по языку
GET	/search	Страница семантического поиска
GET	/dashboard	Аналитический дашборд

Метод	Путь	Назначение
GET	/sources	Управление источниками
GET	/register	Форма регистрации
POST	/register	Обработка регистрации
GET	/login	Форма входа
POST	/login	Обработка входа
GET, POST	/logout	Завершение сессии
GET	/cabinet	Личный кабинет

Программный интерфейс (REST API, JSON)

Блок «Личный кабинет» (требуют авторизации)

Метод	Путь	Назначение
GET	/api/me/session	Состояние сессии и данные пользователя
GET	/api/me/folders	Список папок кабинета
POST	/api/me/folders	Создание папки
DELETE	/api/me/folders/<id>	Удаление папки
GET	/api/me/saved	Сохранённые ресурсы
POST	/api/me/saved	Сохранение ресурса в папку

Метод	Путь	Назначение
POST	/api/me/saved/<id>/move	Перенос записи
DELETE	/api/me/saved/<id>	Удаление сохранённой записи
GET	/api/me/export	Экспорт в BibTeX

Блок «Языки и статистика»

Метод	Путь	Назначение
GET	/api/languages	Список языков
GET	/api/language/<name>	Детализация языка
GET	/api/stats	Сводная статистика
GET	/api/chart/tiobe	Данные для диаграммы TIOBE
GET	/api/chart/github	Данные для диаграммы GitHub

Блок «Семантический поиск и публикации»

Метод	Путь	Назначение
GET, POST	/api/search	Семантический поиск
POST	/api/search/save_external	Сохранение внешнего источника (SourceDocument)
POST	/api/search/save_fact	Сохранение факта (KnowledgeFact)

Метод	Путь	Назначение
POST	/api/sources/add	Добавление источника в онтологию
GET	/api/sources/language/<name>	Публикации по языку

Блок «Обновление данных»

Метод	Путь	Назначение
POST	/api/integration/update	Запуск обновления метрик
GET	/api/integration/status	Статус последнего обновления

ПРИЛОЖЕНИЕ Д

Материалы функционального тестирования и контрольные запросы для оценки поиска

Д.1. Автоматизированные smoke-тесты

Реализованы в файле tests/test_smoke.py. По результатам последнего прогона все 9 тестов пройдены успешно. Полный вывод команды pytest приведён в основном тексте (раздел 4.2).

Идентификатор	Проверяемый маршрут / сценарий	Статус
test_index_ok	GET /	Пройден
test_languages_page_ok	GET /languages	Пройден
test_search_page_ok	GET /search	Пройден
test_dashboard_page_ok	GET /dashboard	Пройден
test_api_languages_json	GET /api/languages	Пройден

Идентификатор	Проверяемый маршрут / сценарий	Статус
test_api_language_python	GET /api/language/Python	Пройден
test_api_stats_json	GET /api/stats	Пройден
test_api_me_folders_requires_auth	GET /api/me/folders (без сессии) → 401	Пройден
test_api_search_post_json	POST /api/search (keywords)	Пройден

Д.2. Ручные тест-кейсы (примеры из полного набора)

Полный перечень ручных сценариев (___ из ___ пройдено) вынесен в отдельную таблицу в архиве с исходными кодами. Ниже — несколько характерных примеров.

ID	Предусловие	Действие	Ожидаемый результат	Статус
ТС-01	Приложение запущено	Открыть /languages	Отображается список языков, нет ошибок	Пройден
ТС-02	Открыта карточка Python	Проверить наличие метрик TIOBE/ GitHub	Поля tiobeRank, githubStars заполнены	Пройден
ТС-03	Авторизованный пользователь в кабинете	Создать папку «Избранное», сохранить туда	Ресурс появляется в папке, экспорт работает	Пройден

ID	Предусловие	Действие	Ожидаемый результат	Статус
		результат поиска		
ТС-04	Форма добавления публикации	Отправить форму без заголовка	Сообщение об ошибке, граф не изменён	Пройден
ТС-05	Запущено обновление с неверным GitHub-токеном	Нажать «Обновить метрики»	Сообщение об ошибке, приложение продолжает работать	Пройден

Д.3. Контрольный набор запросов для оценки качества поиска

Использован набор из 12 запросов, описанный в разделе 4.3. Эталонные URI определены на основе содержимого RDF-графа. Для каждого запроса фиксировалось попадание эталона в топ-3 выдачи.

Таблица Д.3 – Результаты оценки поиска (Success@3)

№	Запрос	Эталон (ожидаемый URI)	Попадание в топ-3
1	Python	:Python	Да
2	JS	:JavaScript	Да
3	C++	:CPlusPlus	Да
4	C#	:CSharp	Нет (требуется доработка)

№	Запрос	Эталон (ожидаемый URI)	Попадание в топ-3 нормализации)
5	книги по Java	:Book + :aboutLanguage :Java	Да
6	документация Python	:Documentation + :aboutLanguage :Python	Да
7	статьи 2020	:Article + :yearPublished 2020	Да
8	языки с ООП	:Python / :Java / :CPlusPlus	Да
9	функциональные языки	:Haskell	Нет (избыточное расширение)
10	Java фреймворки	:Framework от :Java	Да
11	Go	:Go	Да
12	Python и JavaScript	:Python или :JavaScript	Да

Итог: $\text{Success@3} = 10 / 12 = 0.83$

Примечание: значения следует заменить на фактические, полученные в вашем эксперименте.

ПРИЛОЖЕНИЕ Е

Скриншоты пользовательского интерфейса

Разместите рисунки Е.1–Е.п (качество не ниже читабельного для печати А4):

- Е.1 — главная страница (/).

- **Е.2** — список языков ([/languages](#)).
- **Е.3** — карточка языка ([/language/<name>](#)).
- **Е.4** — страница семантического поиска и пример выдачи ([/search](#)).
- **Е.5** — дашборд ([/dashboard](#)).
- **Е.6** — пример ответа в Postman/браузере для GET [/api/languages](#) (по желанию).

Под каждым рисунком: «**Рисунок Е.к** — ...» и пояснение в 1–2 предложения.